



DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis

Design and Implementation of ARM-CCA compatible Unikernel

Johanna Latzel





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis

Design and Implementation of ARM-CCA compatible Unikernel

Design und Implementierung eines mit ARM-CCA kompatiblen Unikernels

Author:	Johanna Latzel
Supervisor:	Prof. Dr. Pramod Bhatotia
Advisor:	Patrick Sabanic
Submission Date:	04.05.2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 04.05.2026


Johanna Latzel

Acknowledgments

First, I would like to thank my advisor, Patrick Sabanic, for providing support and technical expertise throughout this project. He was always available to answer questions, provide feedback, and offer words of encouragement.

Second, I would like to thank Masanori Misono, whose feedback significantly improved the quality of this work.

Next, I would like to thank Prof. Dr. Pramod Bhatotia for giving me the opportunity to work on this topic and for fostering a great team at the System's Research Group. Many members of the chair generously contributed their time and knowledge to this project.

Lastly, I would like to thank my parents and my sister – I couldn't have done it without you.

Abstract

Large Language Model (LLM) inference requires an immense amount of data processing. Most commonly, cloud computing is employed to meet the high demands for computing power, although on-device inference is a steadily growing approach. In both cases, sensitive data is being processed in insecure, potentially malicious environments. This poses a threat to both the user’s private data and the expensively developed models.

To protect sensitive data against hostile cloud providers or end-users, we present an end-to-end system for confidential LLM inference. We leverage hardware-based confidential computing to fortify the service against an untrusted environment, including the host OS, hypervisor, and other applications. Adapting the unikernel architecture to Arm CCA allows us to execute a performant, specialized libOS within an isolated environment. A custom shared memory channel enables fast on-device communication by limiting the number of exits from the trusted execution environment.

We demonstrate the capabilities of our system by testing on OpenCCA, a framework that allows accurate evaluation despite the lack of available hardware. We achieve secure LLM inference with an overhead of 20% to 2%, decreasing with an increasing number of requests. Our unikernel system achieves $3.5\times$ faster boot than the Linux baseline.

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
1.1 The Case for Confidential Computing for LLM Inference	1
1.2 State of the Art and Research Gap	2
1.3 The Approach	3
2 Background	5
2.1 Confidential Computing	5
2.1.1 Problem Statement and Goals	5
2.2 Arm Confidential Compute Architecture (Arm CCA)	7
2.2.1 Code Execution and Data Isolation Worlds	7
2.2.2 Memory management	9
2.2.3 Realm Execution Processes	9
2.2.4 Realm Attestation and Measurement	11
2.2.5 Device Assignment	11
2.3 Unikernel	12
3 Overview	13
3.1 System Overview	13
3.2 System Model and Workflow	14
3.3 Threat Model	14
3.4 Key Challenges	15
4 Design	17
4.1 Lightweight Realm OS	17
4.2 Realm to World Communication	18
4.2.1 Device Access	18
4.2.2 Shared Memory Communication	19
4.3 Realm Attestation and Key Exchange	20

5	Implementation	22
5.1	Prototype Scope	22
5.2	Software Stack	23
5.3	Arm CCA compatible Unikraft	24
5.4	Shared Virtio Device Access	26
5.5	Shared Memory Communication Channel	28
5.5.1	VMM Shared Memory Backing	28
5.5.2	Slot Based Communication Channel	28
5.6	Inference Service Communication Protocol	29
5.6.1	Model Owner Communication	31
5.6.2	Client Communication	31
6	Evaluation	32
6.1	Experiment Setup	32
6.2	Realm versus Normal World Unikernel Performance	33
6.2.1	Methodology and Measurements	33
6.2.2	Results	34
6.3	Light-weight OS versus Full OS Performance	38
6.3.1	Methodology and Measurements	38
6.3.2	Results	39
6.4	Shared Memory Channel versus VirtIO Device Access	40
6.4.1	Methodology and Measurements	41
6.4.2	Results	41
7	Related Work	43
8	Conclusion	45
9	Future Work	46
	Acronyms	47
	List of Figures	50
	List of Tables	51
	Bibliography	52

1 Introduction

Information technologies such as big data and cloud computing have been on the rise for over a decade and show no sign of slowing down [Has+15]. On the contrary, the AI sector’s unending investment growth promises even more demand for the processing of vast amounts of private data in the future [Mas+25]. The growing momentum of AI goes hand in hand with a growing need for secure data processing. The need for data-in-use protection has driven the adoption of confidential computing (CC) as a widely adopted measure to secure workloads. Instead of relying on an outside party to properly secure the remote computing environment, CC enforces security using the platform as the trust anchor. A study shows that 75% of 600 IT leaders have adopted CC, making it a vital field of research [Bue25].

1.1 The Case for Confidential Computing for LLM Inference

In terms of flexibility, scalability, and portability, public cloud is the leading technology in the field. However, using public cloud requires trusting the cloud provider, both not to tamper with the data itself and to prevent intrusion from the outside. Processing private client data on shared infrastructure requires protecting it. Failing to do so not only harms the client but also constitutes a breach of regulations which will result in a high fine for the responsible party. Furthermore, intellectual properties, such as AI models, are often deployed in untrusted environments [Amb24].

Since sensitive data is being processed on an untrusted platform, high-privilege parts of the system pose a threat. In the case of Virtual Machines (VMs), specifically, the hypervisor has the ability to access and modify VM data at its discretion. Direct Memory Access (DMA) peripherals and other user applications are additional potential attack vectors. CC provides an isolated environment to the VM, thereby transforming it into a Confidential Virtual Machine (CVM). CVM data and workloads are protected from tampering and unauthorized access [Kuz23].

LLM Inference in Confidential VMs. In the public cloud, the use of a CVM protects the confidentiality and integrity of user inference requests. The user sends the encrypted request to the CVM, where it is decrypted and processed securely under CC protection.

This protects user data against the hostile environment, but leaves it vulnerable to potential misuse within the VM. There are no guarantees that the data will not be retained or used to infer personal attributes about the user [Sta+24]. This threat to user privacy can be mitigated through on-device LLM inference, where private data never leaves the confines of the user-owned hardware [Wan+25]. However, this puts the model at risk of a malicious user and host system. CC protection on the end-user device is needed to keep the model protected. An end-user device compatible, resource-efficient CVM is required to enable a secure inference service.

1.2 State of the Art and Research Gap

Hardware-based Confidential Computing Support. Three main vendors propose a hardware-based solution for creating trusted execution environments (TEEs) required by CC. These propositions are in different stages of research, development, and availability. Intel Software Guard eXtensions (SGX) offers process-level isolation, while Intel Trust Domain eXtensions (TDX) supports CVM-level isolation. AMD Secure Execution Virtualization (SEV) is another VM based solution, which has undergone several enhancements, including Secure Nested Paging (SNP), to add stronger memory protection against hypervisor attacks. Since 2004, Arm has offered Arm Trust Zone to support trusted applications. To join the system-level isolation required for CVMs. Arm proposed Arm Confidential Compute Architecture (Arm CCA) in 2021. Hardware support for Arm CCA is unavailable as of yet. However, academic research and the development of software support are underway [Fen+24; ZA25; DL21]. Since Arm architecture dominates the mobile device market, Arm CCA is especially interesting for on-device applications on smartphones [Che25].

Confidential Computing and Unikernels. Employing CC drastically improves the security of any system, but in turn accepts an unavoidable loss in performance [Xu+24; Hua+24]. Unikernels are a promising solution for single-purpose systems that offer significant performance improvements while minimizing the attack surface of the VM [Kue+21; Mad+13a]. Joining unikernels with CC combines the small Trusted Computing Base (TCB) and efficiency of a specialized Operating System (OS) with the security of a TEE. Previous work has focused on securing unikernels with SGX [SG18; WH24]. Existing work on a modification of Unikraft [Kue+21] for basic Arm CCA functionality was never adopted for release [Zha23]. Only the Linux kernel, with a large attack surface and a lack of specialization that limits performance, is currently compatible with Arm CCA.

ARM CCA and LLM Inference. Academic research on Arm CCA most often focuses on architecture extensions. CAGE [Wan+26] extends the Arm CCA firmware stack to support confidential accelerator usage. PORTAL [San+25] enables secure device access in general, including Graphics Processing Units (GPUs). On the software application side, GuaranTEE [Sib+24] proposes a framework for providing private machine learning on edge devices. Their main security goal is the protection of the AI model. They use the Linux kernel to execute an end-to-end image-recognition service with a file system-based communication channel. The lack of hardware support limits the scope of the evaluation, as the test environment lacks vital features such as attestation and cannot provide accurate timing measurements.

Research Gap No unikernel support is currently available for Arm CCA, even though they show promise in CC systems. The examination of inference on Arm CCA systems has been limited by a lack of suitable emulation environments. We have seen ongoing research on unikernels employed as CVMs and securing AI with Arm CCA. This leaves a so far unaddressed gap – combining the two approaches to support secure, high-performance LLM inference on Arm devices.

We ask the following research question: *How can we ensure high-performance LLM inference in untrusted environments while protecting model and data confidentiality and integrity?*

1.3 The Approach

To secure the integrity and confidentiality of model and user data, we propose the use of a CVM with hardware-based confidential computing. We choose Arm CCA over its competitors because it is particularly relevant for the growing on-device inference use case, given the dominance of Arm on mobile devices.

Modifying the unikernel architecture for Arm CCA compatibility enables us to explore the advantages a specialized OS offers for secure LLM inference. We leverage its modularity to reduce the system’s attack surface, adding another layer of security. Furthermore, we benefit from the improved throughput and overall performance of a unikernel. We examine various communication options between the CVM, the host, and external entities. To address the high cost of CVM exits, we design a high-performance shared-memory communication channel and compare it with a paravirtualized device. The system is easily extendable for future improvements and adaptable to different runtime environments. It requires no changes to the trusted Arm CCA firm- and hardware stack. Our design results in an end-to-end service prototype for secure inference on hostile devices.

Evaluation. To evaluate our prototype’s performance as accurately as possible, we use the OpenCCA [BS25] framework to overcome the lack of actual Arm CCA hardware. We provide a detailed breakdown of CC-based versus insecure execution to examine the overhead of the secure system and identify time-intensive areas to inform future improvement decisions. To confirm the advantages of a unikernel CVM, we contrast performance and memory impact with the full Linux OS. Virtual Input-Output (VirtIO) device access is compared with our shared memory communication design.

Contributions.

- We propose a light-weight adaptable unikernel compatible with Arm CCA.
- We design a shared-memory communication channel to enable message exchange between the host and guest.
- We create an end-to-end, secure LLM inference service workflow from startup through attestation to model loading and inference.
- We evaluate the prototype to measure the advantages and disadvantages of the employment of unikernel technology in Arm CCA using OpenCCA [BS25].

2 Background

Our design aims to maximize security and performance by integrating modern confidential compute architecture with task-specialized unikernels. The combination of these two concepts requires a fundamental understanding of the individual. In the following, we introduce both topics in the necessary detail.

2.1 Confidential Computing

Information security calls for the protection of data in all its states. Security mechanisms need to meet different challenges and requirements depending on the state of data they target. Securing data in cloud storage or databases (data-at-rest) and securing data during network transfer (data-in-transit) are long-established areas of research. This has led to standardized and universally adopted solutions, like disk encryption or the TLS protocol [ZA25]. With the rise of information technologies, like big data and artificial intelligence, the deployment of workloads including sensitive data or intellectual property on untrusted machines has become a necessity. As a result, the protection of data actively being processed (data-in-use) on cloud or edge devices has become of critical importance. Confidential computing aims to fill this gap - it is based on, but not limited to, the application of hardware-based TEEs to isolate and secure data in use [Fen+24].

2.1.1 Problem Statement and Goals

Confidential computing (CC) is not the only approach to securing data in use; homomorphic encryption and Secure Multi-Party Computation (SMPC) share the same goal [Bue25]. However, CC puts its focus on a particular application framework with specific challenges and requirements. An overview of the CC problem statement is presented in Figure 2.1.

At the center is the untrusted environment, wherein an untrusted entity is processing sensitive data. This platform can range from a remote Cloud Service Provider (CSP) to an end-user edge device. In both cases, an untrusted and potentially malicious entity has immediate access to the data-in-use. It has full control over the software running on the platform, as well as the firmware and hardware in use. Potential attack vectors

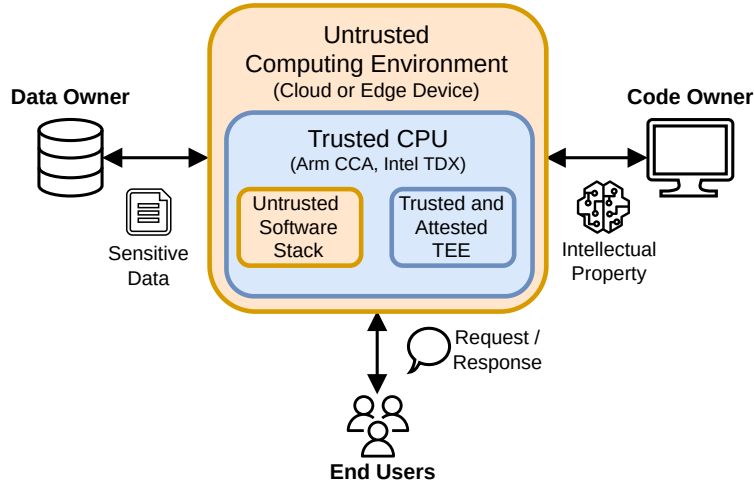


Figure 2.1: Overview of Confidential Computing Environment and Participants. Figure informed by [Fen+24].

include a compromised host OS and hypervisor, as well as untrusted applications. The code owner provides the untrusted entity with intellectual property, for example, an LLM. The data owner supplies the necessary sensitive information, such as private health data. Valuable data enters an untrusted environment, where it must be protected while in use [Fen+24].

Data security requires a root of trust. CC designates the hard- and firmware of the TEE as the trust anchor, relying on the trustworthiness of the hardware manufacturer. Outside entities need to trust only the TEE, as well as the code running inside of it [Fou23].

Thus the specific requirements and goals for CC are as follows [Bue25; Fen+24]:

- **Isolation.** The isolation of sensitive data and code from the untrusted OS and hypervisor must be ensured. TEEs supply hardware-based isolation.
- **Confidentiality and Integrity.** Data must be protected from unauthorized access and modification. Crucially, availability is out of scope for the CC model. Availability can be indirectly improved, however, a compromised host cannot be prevented from denying service.
- **Attestation.** All outside entities can request cryptographic proof of the TEE, to enable third-party verification of its security state.
- **Performance.** CC is intended for services that process large amounts of data, such

as generative AI, so performance requirements are high. Furthermore, platform providers such as CSPs are expected to dynamically create and destroy workloads.

2.2 Arm Confidential Compute Architecture (Arm CCA)

Arm first introduced the TEE Arm TrustZone in Armv7. Their newest version, Armv9.2, includes CC support and is designed with AI applications in mind. Arm CCA provides the necessary hardware, the Realm Management Extension (RME), and firmware, the Realm Management Monitor (RMM) and TF-Monitor, to secure workloads in untrusted environments. Figure 2.2 shows an overview of the Arm CCA architecture. No commercial Arm CCA devices are available yet. The following introduction is mainly based on the official documentation [Arm25b].

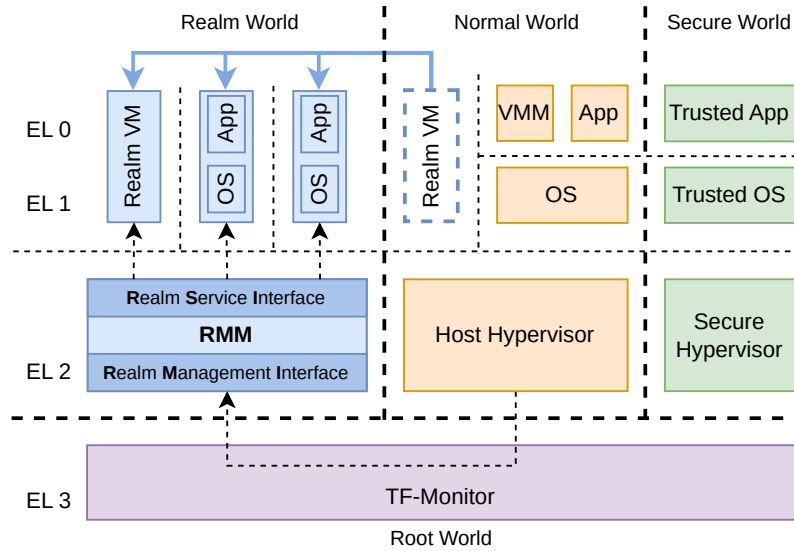


Figure 2.2: Arm CCA Overview. Figure informed by [Arm].

2.2.1 Code Execution and Data Isolation Worlds

Worlds describe the security state of a Processing Element (PE) and its physical address space (PAS). Depending on the security state, the PE is granted or denied access to the PAS of other worlds. Arm TrustZone separated the platform into two worlds – Normal and Secure. To provide CVM support Arm CCA adds two further worlds – Realm and Root. Table 2.1 visualizes the access rights of the different worlds.

Normal World. The Normal world is the basic non-secure world running unprotected workloads. It is not allowed to access any other PAS but its own. The host OS and hypervisor are a part of this world. The host hypervisor has the power to dynamically allocate and destroy resources for the Realm world. However, it can never observe data or code execution within the Realm.

Secure World. The Secure world is isolated from the Normal world. It contains only trusted OS and applications. Since all code entering the Secure world must be trusted, it is unsuitable for CVM execution. Since the Realm world is completely isolated from the Secure world, it is irrelevant for our purposes.

Root World. The Root world is a newly introduced world running at the highest privilege execution level with its own PAS. It is responsible for privileged processes such as trusted boot and switching between worlds. The TF-Monitor is the trusted firmware responsible for communication between the Normal world hypervisor and the Realm world.

Realm World. The Realm world is the designated world for the execution of CVMs, also called Realm VMs or Realm guests. Realm VMs can be created and destroyed dynamically as well as run in parallel. The Realm world includes the RMM. It manages all Realm VM execution and hypervisor instructions. Its responsibilities include Realm creation, resource allocation, and interrupt support. The RMM connects to the host over the Realm Management Interface (RMI), whereas the Realm VM uses the Realm Service Interface (RSI). The Realm Host Interface (RHI) provides a direct communication channel between Realm VM and host. This is intended for requests not involving Arm CCA firmware, for example, provisioning of information during guest boot. The firmware in the Realm world can access the Normal world. This enables shared buffers between Realm VMs and the host.

		Physical address space			
		Normal	Secure	Realm	Root
Security state	Normal	✓	×	×	×
	Secure	✓	✓	×	×
	Realm	✓	×	✓	×
	Root	✓	✓	✓	✓

Table 2.1: Access control between worlds.

2.2.2 Memory management

The four different architectural PASs are mapped onto the actual System PAS. Isolation between worlds is ensured through the Granule Protection Check (GPC). The Granule Protection Table (GPT) assigns each granule of physical memory to a world. The Root world monitor can transfer control of the physical memory by updating the GPT.

A Realm VM access follows this process: The Memory Management Unit (MMU) translates the Virtual Address (VA) into an intermediate physical address (IPA) in the stage-1 translation table. In the second stage, it is translated to the actual Physical Address (PA). Then the GPC is executed to make sure the access is permissible.

The stage 2 translation uses the Realm Translation Table (RTT). The granule size is fixed at 4KB. To denote Normal world memory as shared with the Realm, Arm CCA introduces a new Non-Secure Enable (NSE) bit to the GPT. This is the Most Significant Bit (MSB) of a Realm IPA. Thereby, the address space is divided into the lower Protected IPA - MSB 0 - and upper Unprotected IPA - MSB 1. The transition between unprotected and protected is processed by the RMM and can be requested by the Realm through the RSI [Arm24].

2.2.3 Realm Execution Processes

The Arm CCA architecture incurs significant performance loss in a trade-off for security. The decrease in performance can be traced to two main points of origin.

Realm Startup. The start-up of a Realm guest adds multiple steps to the normal VM preparation process. Figure 2.3a shows the workflow from Realm creation to Realm attestation. While some steps add a static amount of overhead to the boot process, others are significantly impacted by the size of the initial Realm image. RTT Tree creation must create a sufficient number of RTTs to cover the entire mapped Realm IPA. All Realm RAM must be initialized as Realm protected memory. The Realm Population process is the most affected by the initial Realm image size. For each page of image memory, the host must allocate and delegate the granule, then populate it with data and update the Realm Initial Measurement (RIM) [Arm24]. Previous work has pointed out this increase in overhead related to image size [Sib+24]. The termination of a Realm guest equally suffers a performance penalty.

World Switches. Related work also showcases a significant overhead for Memory-Mapped IO (MMIO) heavy workloads [Xu+24; Hua+24]. Interrupts, timers, and MMIO emulation all lead to a switch to the Normal world. As Figure 2.3b shows, each Realm

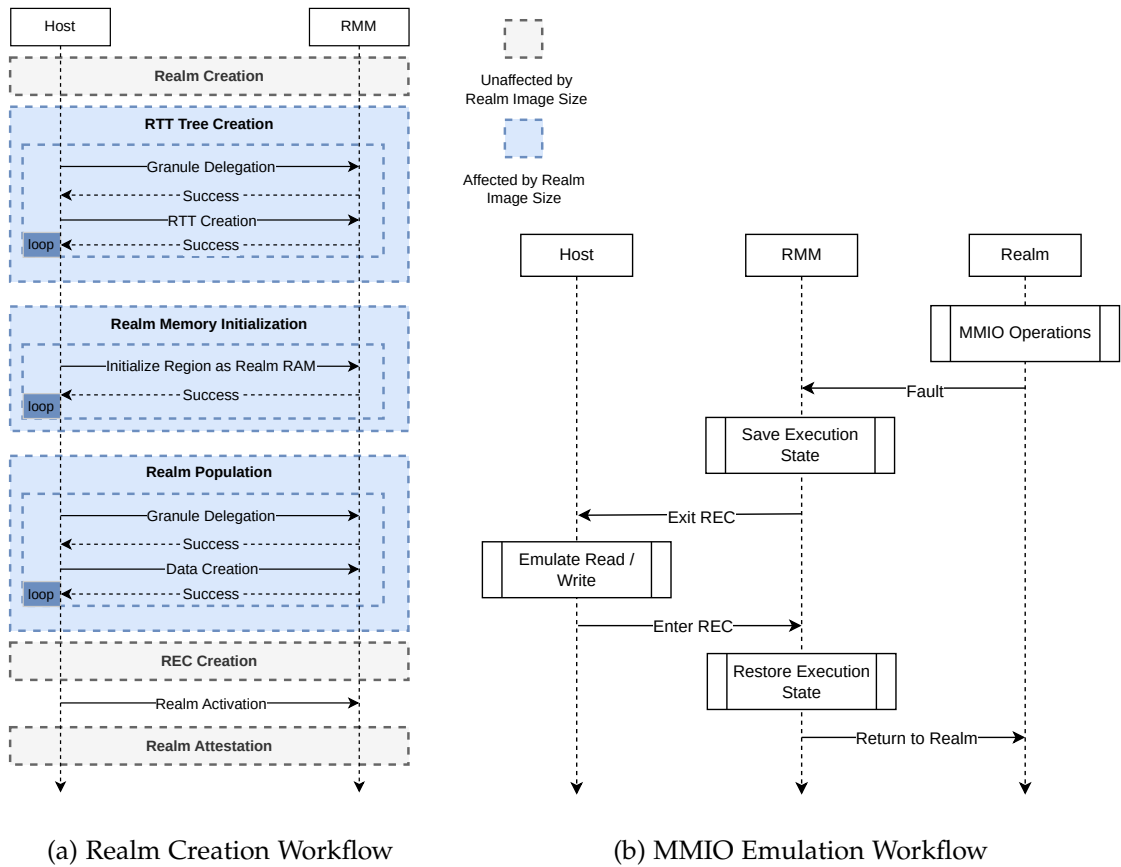


Figure 2.3: Realm Creation and Realm Exit Workflows. Figure informed by [Arm24].

exit incurs an individual overhead penalty to save and restore the Realm Execution Context (REC) and switch between different Execution Levels.

2.2.4 Realm Attestation and Measurement

Realm attestation is a vital part of the Realm guest execution process. Without attestation, there is no guarantee that the VM is correctly isolated from the Normal world. A successful attestation proves that the CVM is running on a trusted Arm CCA platform and that the Realm is in a secure state. It is best practice to follow the Remote Attestation Specification (RATS) for correct attestation.

Attestation Report. The attestation report must prove the validity of hardware and firmware. It is split into two parts. The platform token includes the Arm CCA platform identity, lifecycle state, and measurements of the trusted firmware. Verification of these elements confirms the trustworthiness of hardware and firmware. The Realm token contains the Realm measurements, the attestation challenge and Realm Attestation Key (RAK). The verification of this token confirms the secure boot of the Realm VM. The Realm can request a report at any time to prove the validity of the platform and code [Arm24].

Realm Measurement. Realm Measurement includes two distinct capabilities, Realm Initial Measurement (RIM) and Realm Extended Measurement (REM). The RIM is initialized over Realm parameters and then extended whenever significant changes are made during Realm construction. This includes adding memory, creating a REC, or modifying the protection status of a page. The RIM becomes immutable once the Realm is active. As a consequence, all changes made to the Realm state from within the guest are not reflected in the RIM. The REM can be used to measure changes made during the lifetime of a Realm guest. The measurement is invoked with the RSI [Arm24].

2.2.5 Device Assignment

Arm CCA proposes RME-Device Assignment (DA) under CCA to enable assigning a device to an individual Realm. After device attestation, the device has access to Realm contents. The hypervisor and other Realms are prevented from controlling the device. However, this feature still sees no support in terms of emulation and testing.

Lacking a secure device assignment, devices and all the associated memory areas must be shared with the Normal world. Therefore, all device communication of sensitive data must be properly secured in transport [Hua+24].

2.3 Unikernel

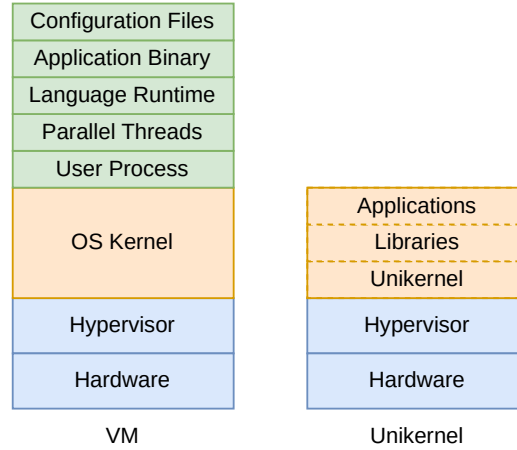


Figure 2.4: Comparison of Traditional Virtual Machine and Unikernel Software Layers.
Figure informed by [Goe+18] and [Mad+13a].

Unikernels are "single-purpose appliances that are compile-time specialised into standalone kernels" [Mad+13a]. A full OS usually splits the address space into kernel and user space. This provides memory protection from user applications, but necessitates context switching between both address spaces. Since unikernels are only intended to provide a single functionality as a VM, they are isolated from host memory already. Therefore, they can have a single memory address space for kernel and application, eliminating all needs for context switches and improving performance significantly. Unikernels are based on library OS (libOS). Unlike a monolithic OS, all features, from device drivers to the scheduler, are libraries that are linked directly with the application. This enables the configuration of a unikernel to include only what is required for its specialized purpose. The exclusion of superfluous features naturally leads to a small image size and minimizes the application's attack surface [Mad+13a]. Unikernels have proven to beat traditional VMs in boot time, resource utilization and I/O performance [Kue+21].

3 Overview

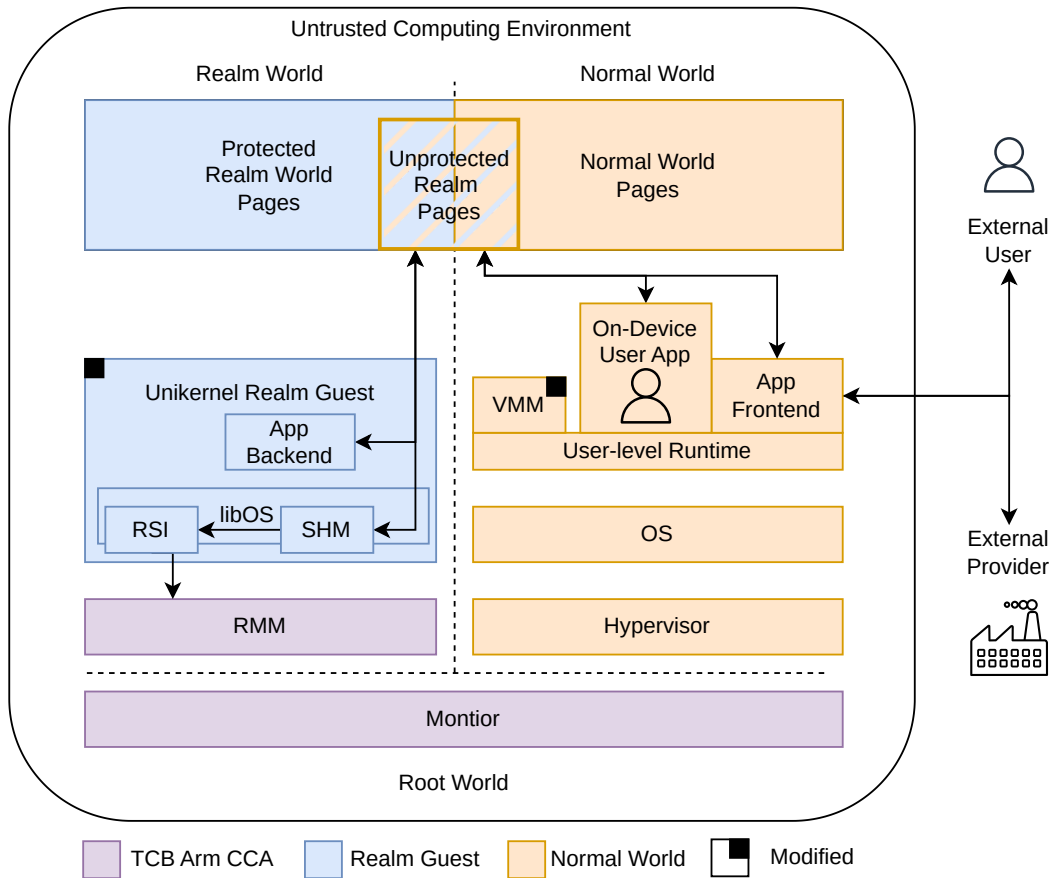


Figure 3.1: System Overview.

3.1 System Overview

We present the design of our end-to-end secure LLM inference prototype. The system must comply with three main design goals. Firstly, we guarantee confidential computa-

tion by ensuring that all workloads are isolated in an Arm CCA Realm guest. Secondly, we minimize the TCB and attack surface of the guest by adapting a unikernel to the Arm CCA architecture. This requires additions to the unikernel libOS. We implement the RSI in a dedicated driver and modify the boot process to enable secure startup and attestation. Lastly, all communication between external entities and the Realm guest must be secured. We add a configurable shared memory communication channel to the unikernel. This enables performant communication between the Realm and the Normal world. The host-side Virtual Machine Monitor (VMM) must make the required memory available to user applications. As a comparison, we employ a rudimentary implementation of VirtIO device access in the Realm. To secure the data transmitted over shared memory, we instrumentalize the Realm attestation protocol to establish shared key material and protect messages against tampering and unauthorized access, while simultaneously attesting the secure state of the Realm. Figure 3.1 shows an overview of the system. We modify only the unikernel Realm guest and Normal world VMM.

3.2 System Model and Workflow

We consider two external parties. Our external provider includes the function of code and model owner. The external user is the client sending inference requests. We consider both a remote and an on-device user for our design.

First, the code provider transfers the Realm image to the untrusted host. The host then triggers the boot of the VM in the Realm. After the boot, the Realm guest opens the shared memory communication channel. To assure the secure state of the Realm guest, the model owner requests attestation and simultaneously the exchange of key material to enable secure communication with the Realm. All communication following this exchange is confidential, integrity protected, and authenticated. This allows for the secure transfer of the LLM to the Realm. Once the model has been loaded, the inference service is ready for client connections. Each end-user also has to complete the attestation protocol to enable usage of the untrusted shared memory channel. After this is completed, inference requests and responses can be exchanged between the realm and the end-user securely.

3.3 Threat Model

The entire Normal world is considered potentially malicious by the code and model owner. This includes the hypervisor, DMA-capable peripherals, and all user-space applications, including an on-device client. All such components must be prevented

from accessing any internals of the model. Client data sent to the inference service must also be protected against Normal world access. The client and host are assumed to trust the code owner – the Realm image functions correctly if the secure boot in the Realm succeeds and does not attempt to compromise the host. Both client and provider consider the Arm CCA hardware and firmware as trustworthy. As a consequence, requesting and verifying the Arm CCA attestation report suffices to confirm correct operation and rule out malign tampering by untrusted components with the Realm. Realm memory and execution context are safe from other Realms and the Normal world. Since the host has full control over resource allocation, Arm CCA cannot make any guarantees on availability. The hypervisor or other Normal world components intentionally interfering with service execution is therefore excluded from our threat model. Furthermore, our attacker model does not consider side-channel attacks. All communication between Realm and external parties is encrypted and thereby assumed to be protected in transit [Arm21].

3.4 Key Challenges

Challenge #1: Small and Performant Realm Image A traditional operating system provides applications with a large variety of functionality to ensure ease of use and portability of applications. When using the OS as a VM with a single purpose, such as an LLM inference service, many of the provided capabilities remain unused and become redundant. As a consequence, the memory footprint is larger than necessary. This impacts boot time under any circumstances, but especially so in a Realm. While the Linux kernel can be customized to exclude certain subsystems, it is not designed with this purpose in mind. Furthermore, it is limited by the need for a separate kernel and user space, which introduces unnecessary overhead in the form of context switches. For our system, we utilize a libOS to provide only the essential functionality and unify the address space. Using a modular OS, we create a small Realm image and maximize performance. This minimizes and balances out the performance cost inherent to a TEE.

Challenge #2: Secure Communication For the purpose of LLM inference protected within a Realm, we require a secure method of communication between the Realm and external entities. Secure device assignment is not yet supported by any development stack and can therefore not be utilized. VirtIO devices provide basic functionalities like network access or file system mounting. However, MMIO heavy applications suffer the highest performance penalties in a Realm environment, as many Realm exits incur a heavy performance overhead.

As an alternative, we present a communication channel that utilizes a shared memory

region between the Realm and the Normal world for efficient communication without triggering a world switch. Communication is secured with an attestation protocol that utilizes the Realm attestation report to create a secure channel between the required parties.

Challenge #3: Adaptability Making unikernel Arm CCA support viable for future systems requires easy adaptation to different functionalities and VMMs.

We support the unikernel libOS to inherently provide adaptability in terms of functionality, guaranteeing that our Arm CCA kernel design and shared memory communication can, in the future, be adapted for purposes other than LLM inference. We aim to provide conformity to multiple VMMs and available development environments for Arm CCA.

4 Design

This chapter explains and gives motivations for the choices made in designing an Arm CCA secured LLM inference service.

4.1 Lightweight Realm OS

The security gained by the execution of a Realm CVM necessarily incurs a performance loss. The origins of this decrease in performance are explained by the Arm CCA architecture and design, as we have discussed in Section 2.2.3. One of the processes that is most impacted in terms of efficiency is the Realm startup and shutdown. The delay is not a static increase in latency, but is affected heavily by the size of the VM image. Efficient LLM inference demands a short Time To First Token (TTFT) from a cold start. On-device inference applications especially require elastic start-up and shutdown to seamlessly switch between providing the service when needed and shutting down to free up processing power.

Arm CCA compatible Unikernel

Unikernels are the perfect solution to address Challenge #1. They minimize the image size, thereby lowering the TTFT. Their libOS can be specifically configured to only include necessary code and functionalities in the Realm image. Whereas a full OS such as Linux will always contain what is ultimately redundant code.

To make a unikernel Realm compatible, it must fulfill the following requirements. Firstly, it must fully support the Realm Service Interface (RSI) used to communicate with the RMM. The RSI can then be utilized to modify the unikernel boot process and make the memory initialization Realm compliant. Furthermore, we must provide functionality for dynamically sharing and protecting regions of memory as well as generating the Realm attestation report.

This design choice also addresses Challenge #3. We employ a configurable libOS to enable repurposing the unikernel for different functionalities and extend the required support to all compatible VMM boot processes.

4.2 Realm to World Communication

We address the first part of Challenge #2 – Communication between the Realm and outside entities – in this section. Applications running securely in the Realm serve no purpose when they cannot receive data from and communicate their results to the outside world. When information enters or leaves the trusted Realm environment, security must be maintained. There are two main communication scenarios. In an on-device inference scenario, the response must be communicated back to the Normal World. Remote requests, whether for inference or model management, require communication between the Realm and a completely external entity.

Our Realm-to-world communication must meet two main design goals. Firstly, the confidentiality and integrity of the data must be ensured, and the security of the Realm must not be compromised by incoming data. Secondly, the communication must be as efficient as possible. Since the world switching overhead for I/O operations is punishing, the number of Realm exits must be minimized.

4.2.1 Device Access

Insecure Device Access

The configuration and operation of devices requires access to the VM memory. However, in a Realm, this access is by default prohibited. Attaching a device the traditional way requires making all the necessary memory regions available to the host.

The standard for virtual environments is VirtIO devices, designed for straightforward and efficient device emulation. Supporting devices like network adapters or mounted filesystems is a key factor in making the unikernel extensible and adaptable to other functions. The VirtIO device accessible memory comprises descriptor rings for input and output, pointing to the buffers containing the actual transmission [OAS23]. To enable VirtIO device access, we therefore have to make sure that the bus, descriptor rings, and buffers are all located in shared memory. There are two potential approaches to this problem.

We could set the memory protection dynamically, sharing a region whenever a new buffer is created and unsharing once it is destroyed. This approach comes with two drawbacks. Memory can only be shared on a page-wise basis, meaning our allocated buffers must be a minimum size of one page. This leads to unnecessary memory overhead. Furthermore, sharing a page requires host involvement, leading to a Realm exit that adds to the already heavy MMIO performance penalty.

Instead, we decide to share a large static memory region during startup to avoid the increase in performance overhead during service. It also decreases the risk of the shared memory containing any confidential data, since access to this memory can

be constrained through one allocator. We accept the less flexible management and unused shared memory overhead incurred by this approach. Since all memory touched by devices is shared, ensuring the confidentiality, integrity, and authenticity of the transmitted data must be added on top of the device communication. This can be achieved through standardized data-in-transit security mechanisms.

Secure Device Pass-through

Avoiding the added transport security layer requires a secure device assignment to a Realm. While Arm CCA proposes such a mechanism, it is not yet available in any of the development environments. Secured device access for specific devices is an active topic of research in the Arm CCA community. All of the thus far explored approaches require extensive changes to the TCB. Modifying the TCB is outside of our scope, but we discuss possible design options for secure accelerator access in chapter 9.

4.2.2 Shared Memory Communication

Instead of VirtIO devices, we can build our own communication channel between Normal and Realm World, leveraging shared memory. This approach comes with its own advantages and drawbacks. The primary reason for establishing a shared memory channel is versatility. Establishing our own channel with its own interface can support any kind of data transfer without added drivers or devices. This is especially relevant, since even shared device access is limited in currently available VMM Arm CCA support stacks, as we will see in chapter 5. Furthermore, we expect shared memory communication to trump emulated MMIO in terms of latency and throughput. We can avoid interrupts and expensive Realm exits entirely.

However, these advantages come at a cost. While VirtIO devices easily enable standardized communication protocols, designing our own heightens our development effort and makes this approach less adaptable to other use cases. Another disadvantage is the lack of a direct channel between external entities and the Realm. A network adapter can establish a direct path between the Realm, and for example, the model owner, the VMM in Normal World emulates the device and thereby forwards the connection without any extra components. On the other hand, a shared memory channel only exists between the Normal and the Realm World. To extend the communication to external entities, an additional application in the Normal World must receive the data over the shared channel and forward it to the correct partner. Despite these drawbacks, we decided on a shared memory communication channel for our system in the pursuit of maximizing performance.

In our design, communication flows between three parties. The external client or model

owner sends a message to the remote device. The front-end application in the Normal World receives the message. This message is then loaded into the shared memory communication area. On-device clients can write to the communication channel directly. The backend application in the Realm world can receive the messages without delay and process the request. Responses are sent analogously, from the Realm to the outside world. On one hand, the communication channel must be capable of transmitting several inference requests at the same time to improve throughput and reduce latency. Splitting the shared memory into separate slots enables communication in parallel. On the other hand, we anticipate the transfer of large amounts of data for the model loading process. Therefore, we enable and optimize the transmission of the model by using the entire shared memory region to transfer large payload sizes.

4.3 Realm Attestation and Key Exchange

To complete Challenge #2, we discuss the security of our communication channel. As discussed in section 2.2.4, Realm attestation is required to guarantee the trustworthiness of the platform and the correct isolation of the Realm VM to all external entities. Following the Remote Attestation Specification (RATS), the model owner and clients need to trust an external verifier to attest to the correctness of the Realm report. While one-sided attestation is sufficient for client interactions, the model owner needs to provide attestation to the Realm to invoke the privileged model management functionality. We limit Realm token verification to the Realm Initial Measurement. Since the initial state of the Realm is sufficient to attest security and correctness to all participants, we choose not to use the Realm Extended Measurement.

Attestation and Key Exchange Protocol. Our protocol (Figure 4.1) must satisfy the verification of all Arm CCA guarantees. This is accomplished by generating the attestation report in response to the challenge and leaving validation to an external trusted verifier. In addition, we require the establishment of a secure channel between the Realm and the external party. To satisfy this requirement, we execute a Diffie-Hellmann (DH) key exchange. The public key of the external client is sent to the Realm. The Realm creates the challenge for the report. This consists of the hash of the Realm public key concatenated to the hash of the client public key. Once the report has been verified, the client can trust that they have established a direct connection with the Realm since the client key is bound into the report. As a consequence, the resulting symmetric key enables secure communication between the Realm guest and client. A drawback of this design is the need for Realm attestation at the beginning of each connection.

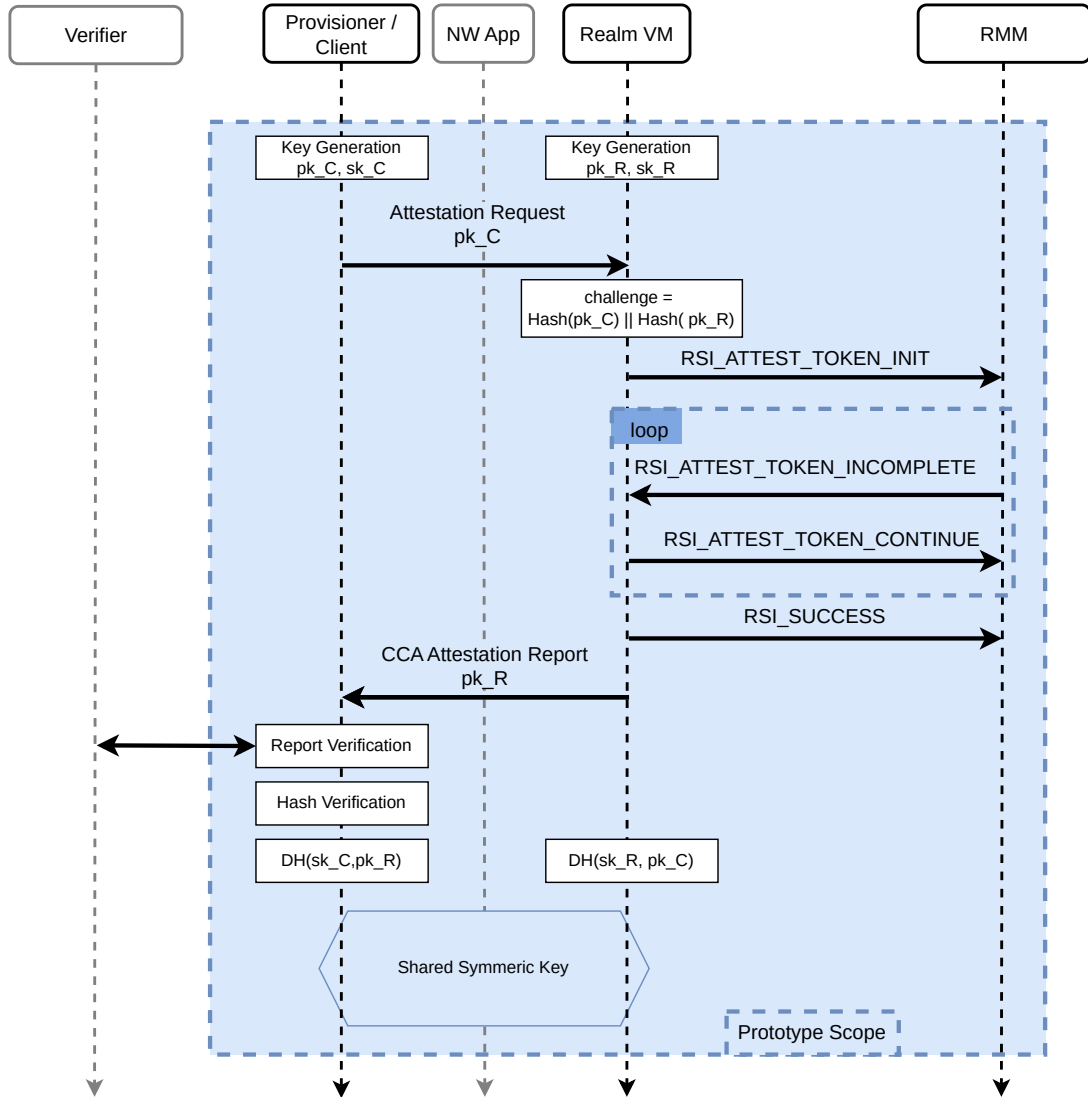


Figure 4.1: Attestation and Key Exchange Protocol.

5 Implementation

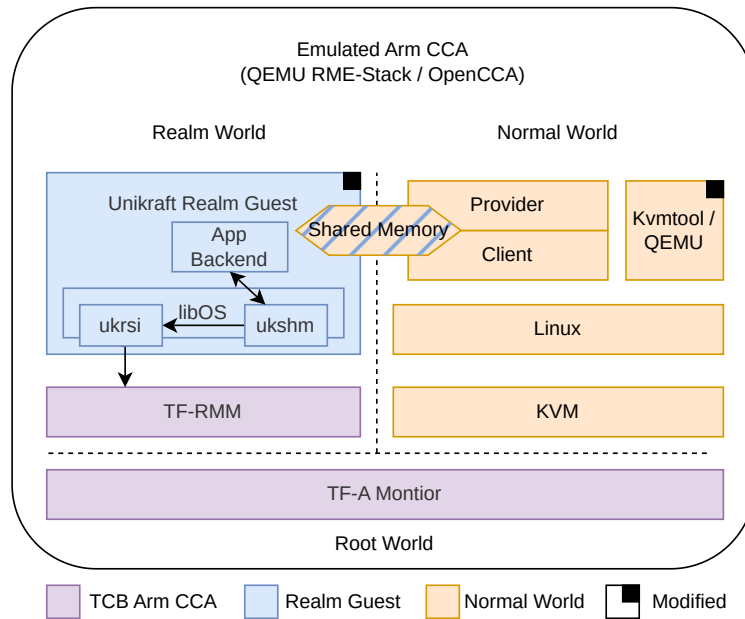


Figure 5.1: Prototype Overview.

5.1 Prototype Scope

The main focus of this work is the impact of the Realm unikernel on the design and performance of LLM inference. Therefore, we limit the scope of our proof-of-work implementation to the essentials. Our end-to-end secure inference service prototype makes concessions to the design by limiting the participating entities to the following:

- **Secure LLM Inference Service:** This includes the Realm guest with the application backend and the shared memory communication channel. We do not implement the frontend forwarding application and instead use direct shared memory access for test purposes.

- **Client:** The client requesting inference service is located on-device in the Normal world. We still encrypt client-to-Realm guest messages to protect the communication from other Normal World entities. To properly isolate on-device inference from other processes, the client itself must be secured. We consider this out of scope in our system.
- **Model Provider / Code Owner:** The model and code owner are the same entity. This entity is simulated in the Normal world. We make this concession since it significantly eases the test setup and since the message transfer between outside entities and the Normal world remains unimpaired by Arm CCA and our design. The Realm image transfer is outside of our scope. The attestation of the model provider and the encryption of the model are not implemented due to the memory limitations of the test environment.
- **Trusted verifier:** Instead of consulting an external trusted verifier, provider and client emulation assume the verification to be successful.

5.2 Software Stack

Runtime Environment. Since the Arm CCA hardware is not available yet, an emulation of the hardware is required for development and testing. There are three main available options. The Fixed Virtual Platform (FVP) is the official ARM emulation platform using Kvmtool as the VMM. However, it lacks critical features such as the generation of the attestation report [Git; Sib+24]. The QEMU RME-Stack is currently still in development and receiving constant updates [Ben]. It allows the execution of Arm CCA on any machine and supports both the Kvmtool and QEMU VMM for launching Realm guests. However, the full emulation comes at a high performance cost.

These problems are addressed by OpenCCA [BS25]. This project provides an open framework to emulate the Arm CCA stack on currently available Arm hardware. It simplifies and accelerates the research and evaluation of Arm CCA components. However, OpenCCA is not currently compatible with the QEMU VMM.

Hypervisor and VMM support. The Kernel-based Virtual Machine (KVM) is the only hypervisor currently supported by Arm CCA resources. Related work has enabled the use of Firecracker as an Arm CCA VMM. However, the resulting code is unavailable [RYL25]. Kvmtool is a minimal VMM implementation. With a smaller, more compact code-base, modifications can be easier [Enb+nd]. QEMU's larger code-base is justified by the many more features and devices it offers [Bel05]. We implement a

unikernel that can be launched with both VMMs, though differences in their inner workings lead to a different range of functionality.

There is currently no support for secure device assignment in Realm, though the QEMU RME-Stack is currently working on adding this feature to their development environment.

Unikraft. The choice of unikernel is mainly influenced by the underlying architecture and software stack. While MirageOS [Mad+13b] supports Arm64 development, its focus on Xen hypervisor support makes it incompatible with our KVM hypervisor development stack. We therefore choose Unikraft, which not only supports Armv8 but also has existing work on extending this support to Arm CCA. In a 2023 GSoC project, Unikraft v0.14.0 was adapted to Arm CCA [Zha23]. However, this version of changes was never officially merged with the Unikraft project. Three years later, the changes are incompatible, both with modern Unikraft v0.20.0 as well as the updated Arm CCA specifications. We use this project as a baseline and modify it for current use. Furthermore, Unikraft’s focus on portability and modularity makes it the preferred option for our use case. We want to leverage its modular OS to minimize our image, add custom libraries to support Arm CCA, and make our system easily extendable for the future [Kue+21].

5.3 Arm CCA compatible Unikraft

We adapt the Unikraft OS to be Arm CCA compatible. The introduction to the Arm CCA software stack lists the changes needed to support a Realm guest [Arm25a]. The required changes were largely completed in the 2023 GSoC [Zha23]. We update individual RSI wrappers and calling procedures to fit the specification, as well as port the changes to the newest Unikraft version.

Realm Service Interface. The RSI allows our Realm guest to request services from the RMM. For this purpose, we must implement wrappers for all relevant Secure Monitor Call (SMC) calls. We add the `ukrsi` library to the Unikraft stack for all Realm-to-RMM communication. We implement version 1.0 of the RSI [Arm24]. Configuring Unikraft for this use requires setting options `CONFIG_ARM_64_FEAT_RME` and `CONFIG_LIBUKRSI`. Option `LIBUKRSI_TEST` activates the RSI test suite.

Boot Sequence. The boot sequence for Arm CCA includes the setup of the RSI for further use. The initialization function calls `RSI_REALM_CONFIG` to receive the configuration for the current Realm and consequently the hash algorithm, Realm

Personalization Value (RPV), and the IPA width. The width of the IPA determines the correct NSE bit. This bit indicates whether a page is Realm-protected or shared with the Normal world. It is therefore necessary to initialize the RSI before any memory management is handled.

As we have seen, the VMM can prompt the initialization of Realm memory as protected during the Realm memory initialization. This can later be proven during the attestation step. However, we follow the best practice to iterate over all described RAM and set this memory to RSI_RAM in the Realm [Arm25a]. We iterate over all Memory Region Descriptors (MRDs) and set the used memory to protected. Memory that should stay unprotected is marked as UKPLAT_MEMRT_REALM. This is relevant for memory setup during the early boot stage, which includes early console setup before normal memory initialization.

Early devices, such as the interrupt controller, also require the host to have access to shared registers. To enable interrupts, the NSE bit must be set to mark the designated page as unprotected and allow host access. The same goes for the initialization of all other shared devices. However, normal device initialization occurs after the page table has been initialized and therefore follows a different pattern. The page attribute PAGE_ATTR_RME_UNPROTECTED is used to set the NSE bit. From now on, setting the page attribute is enough to set the shared address. However, it is not sufficient to share the memory with the Normal world, which still requires a sequence of RSI calls. The entire device region is designated as shared memory during startup, since secure device access cannot be implemented at this time.

Algorithm 1 Set Memory Shared

Require: Virtual address $addr$, number of pages n

Ensure: Memory region marked shared

```

1:  $phys \leftarrow \text{VIRT\_TO\_PHYS}(addr)$ 
2:  $phys \leftarrow phys + n \cdot \text{PAGE\_SIZE}$ 
3:  $base \leftarrow phys$ 
4: while  $phys \neq end$  do
5:    $new\_base \leftarrow \text{RSI\_IPA\_STATE\_SET}(base, end,$ 
      $\text{RIPAS\_RSI\_EMPTY}, \text{RSI\_CHANGE\_DESTROYED})$ 
6:    $base \leftarrow new\_base$ 
7: end while
8:  $\text{SETPAGEATTR}(addr, \text{UNPROTECTED})$ 

```

Protecting and Sharing Memory. A Realm guest can dynamically decide to share memory with the host as required. Shared memory can also revert to being protected

memory. We provide a function for both use cases. Algorithm 1 shows the workflow for sharing memory with the host. First, all affected pages must change their IPA state to `RIPAS_RSI_EMPTY` to mark them as unprotected. The `RSI_CHANGE_DESTROYED` flag allows changing the state of pages that have previously been made inaccessible to the Realm by the host. Then, the unprotected page attribute is set to change the NSE bit. Protecting the memory works analogously.

Attestation Report. The generation of the attestation report requires the chaining of several RSI calls as displayed in Algorithm 2.

Algorithm 2 Generate Attestation Token

Require: Output buffer *addr*, challenge *challenge*[8]

Ensure: Attestation token written to memory at *addr*

```

1: ATTESTATIONTOKENINIT(challenge, size)
2: offset  $\leftarrow$  0
3: while ret  $\neq$  RSI_INCOMPLETE do
4:   (ret, len)  $\leftarrow$  ATTESTATIONTOKENCONTINUE(addr, offset)
5:   offset  $\leftarrow$  len
6: end while

```

5.4 Shared Virtio Device Access

We establish the `ukshm` library as an addition to the Unikraft libOS. The configuration option `CONFIG_SHM_VIRTIO` is responsible for VirtIO device shared memory.

Linux Realm Device Access. Linux solves the device access problem using permanently shared bounce buffers in combination with its DMA-API [Com].

The use of DMA accesses requires VMM handling. As we have seen, Realms have a split guest-physical address space, the bottom half being private and the top half being shared. QEMU and Kvmtool implement Realm memory as a merged view of both halves, where individual pages are either private or shared. When a page is accessed, the KVM strips the NSE bit, which correctly adapts it to the merged VMM view. However, DMA addresses remain untouched by the KVM. The VMM must therefore translate the address to fit the merged address space. QEMU solves this problem by adding an IO Memory Management Unit (IOMMU) memory region on the unprotected top half of the Realm. This redirects DMA access to the merged view. Kvmtool relies on external translation of the memory address. Device drivers must be

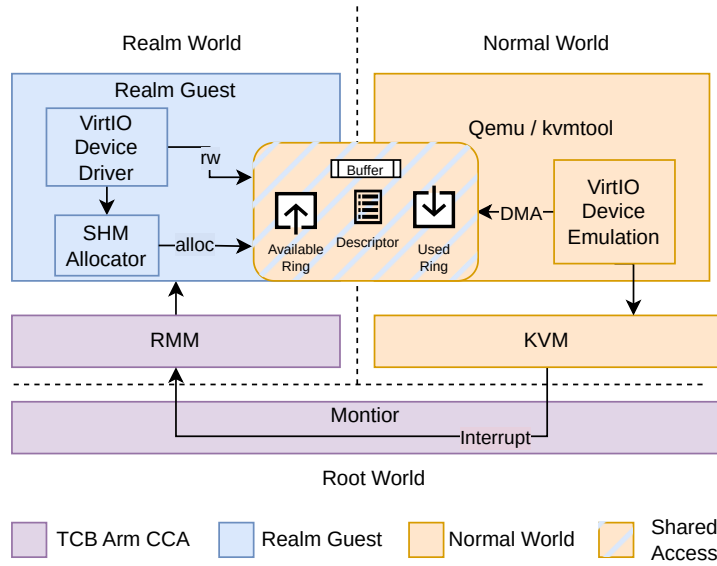


Figure 5.2: VirtIO Device Access

aware of this limitation.

Both Kvmtool and QEMU force the use of the `VIRTIO_F_ACCESS_PLATFORM` feature when creating a Realm guest. The feature indicates that the device can be used even on platforms where access to memory is limited or translated, for example, due to IOMMU translation of physical addresses [OAS23]. Only modern VirtIO devices support these features. Legacy devices are not compatible with the current Linux for Arm CCA implementation. In the Linux guest, this feature triggers the use of the DMA-API combined with bounce buffers. This solves the device access problem, since the entire bounce buffer area can easily be permanently shared on startup. The existing Linux DMA-API bounce buffer integration makes the Realm integration simple and painless.

Proof-Of-Concept Unikraft Implementation. Unikraft offers limited device driver support. VirtIO is available, though only with legacy device configuration. As a consequence, the device will always use the physical addresses as used by the driver, without any further translation step [OAS23]. This requires us to adjust the VMM to correctly handle the translation itself as well as to allow legacy VirtIO in a Realm. We therefore limit support to Kvmtool. To permit access, we must modify Kvmtool to translate addresses correctly. We integrate the translation by masking the NSE bit in the guest-to-host translation step. Unikraft with Kvmtool only supports a MMIO-based bus, further limiting the choice in device access.

In contrast to Linux, lightweight Unikraft contains neither a DMA-API we can hook into nor a bounce buffer region to share. While a sophisticated bounce-buffer implementation would be ideal for our use case, we limit the scope of our work to a proof-of-concept shared VirtIO implementation.

During the early boot stage, a static amount of memory is mapped and shared. An allocator is initialized on the shared memory and assigned to all devices. Since there is no actual bounce buffer, we must ensure that all functions that use the devices either allocate shared memory or utilize an individual small bounce buffer. This is fragile, especially when using external libraries like llama.cpp for inference. We implement this only for the 9p file system mount to enable model loading.

5.5 Shared Memory Communication Channel

We introduce the option `CONFIG_SHM_CHANNEL` to activate the use of our shared memory channel.

5.5.1 VMM Shared Memory Backing

A working shared memory communication channel requires the collaboration of the VMM and the Realm guest. The guest triggers the memory sharing with an RSI call. The VMM is responsible for replacing the physical backing pages with shared memory. To make this channel available to an on-device client, the VMM must further expose the memory to external processes.

QEMU supports this functionality already. Using the `-object memory-backend-file`, we can use a POSIX shared memory backend for the guest RAM. The client process can simply map this file and connect to the communication channel at the right offset. While the entire RAM is backed by the file, only shared Realm memory contents are visible to the Normal world.

Kvmtool does not have native support for file memory backing. We implement the use of file-backed memory for all memory dynamically shared with the Normal world.

5.5.2 Slot Based Communication Channel

As Figure 5.3 illustrates, the shared memory channel is divided into the metadata and payload areas. This allows for the smooth transfer of large amounts of data, such as an LLM. Since the memory is statically shared, it permanently takes up space. The size of the channel must therefore be carefully considered and adapted to the use case. We introduce `CONFIG_SHM_TOTAL_SIZE` to easily change the size of the shared memory region. To support multiple clients simultaneously, both areas are

separated into slots. The number of supported parallel clients can be specified with `CONFIG_SHM_NUM_CLIENTS`.

Communication has the following workflow. First, the client locks an empty slot. The type and length of the message are written to the corresponding metadata slot, while the message content is written to the payload area. Then, the server notifier and active requests semaphore are incremented to signal readiness to the server. Instead of circling through all slots, the server waits on the active requests semaphore, then finds the slot marked by the server notifier. The server writes the response to the request into the metadata and payload slot, then increments the client notifier. Once the communication is finished, the slot is unlocked, freeing it for the next client.

Due to limitations inherent to unikernels, a multi-threaded server cannot be supported. The nature of host-VM shared memory further leads to a lack of kernel signaling support for waking up blocked processes. As a consequence, both client application and server employ spin-locks to await requests and responses.

The shared memory communication channel design is inspired by Ürun [Üru25].

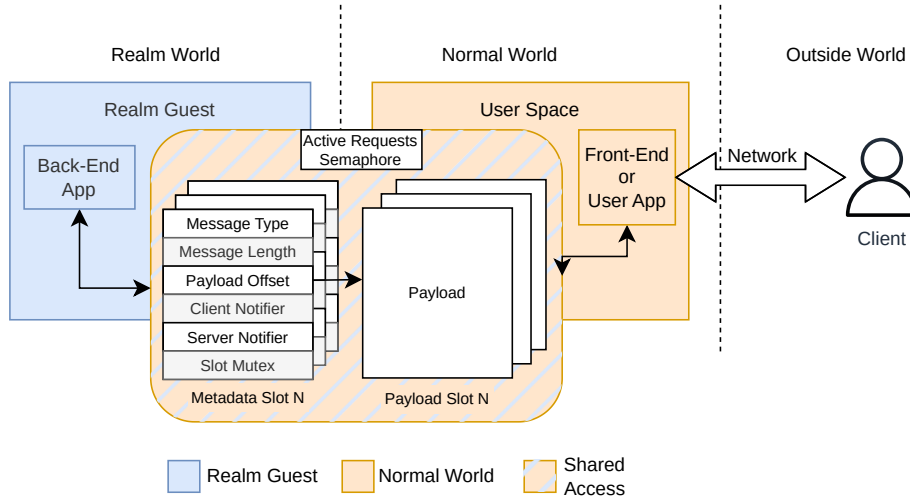


Figure 5.3: Shared Memory Communication.

5.6 Inference Service Communication Protocol

To establish an end-to-end inference service, we support the following interactions.

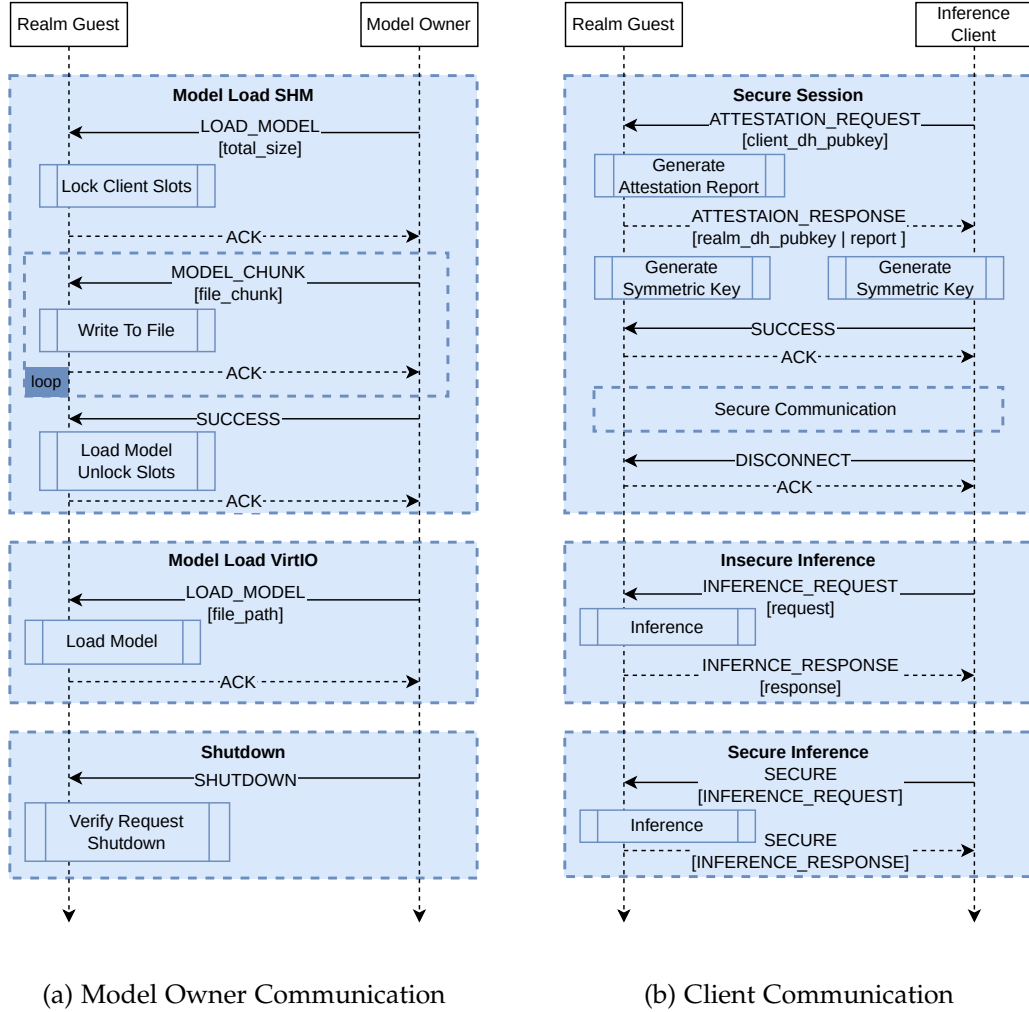


Figure 5.4: Inference Service Communication Protocol.

5.6.1 Model Owner Communication

The model owner can initiate a model load over shared memory or a model load over VirtIO. While loading a model, no client inference requests can be processed. To make use of the entire shared memory region for the model transfer, all other communication slots are locked by the server. The model file is transferred in payload area-sized chunks. Since llama.cpp can only load a model in file format, all chunks must be written back to a Realm-owned file.

In contrast, loading a model over VirtIO, the model file can be loaded directly into the inference engine. However, this prevents encrypting the model for transfer. Secure model loading over a 9p shared file system requires a second mounted RAM file system. A shutdown message causes the Realm to boot down.

5.6.2 Client Communication

The client can initialize a secure session and send inference requests.

Secure Message Transmission. The design of our attestation and key exchange protocol has already been discussed in chapter 4. To implement the encryption of the inference requests and responses, we include the Unikraft `libsodium` library in our configuration. To generate the required randomness, Unikraft offers the `ukrandom` driver, which provides randomness on Arm8.5-A processors that implement the RNG feature. Our test environment does not support this option [Uni]. We instead supply deterministic bytes to the encryption functions for our test prototype purposes.

LLM Inference. We use the llama.cpp inference engine to execute the inference requests. As we cannot support GPU accelerator access, we use solely CPU-based calculations. The Unikraft llama.cpp integration is taken from previous work on LLM-OS [Kam26].

6 Evaluation

6.1 Experiment Setup

Runtime Environment. Since Arm CCA hardware is not yet available, evaluation requires emulating the hardware instead. The Qemu RME-Stack is a powerful development environment, but the nested emulation leads to very slow execution times. FVP does not support all features. Furthermore, it can not deliver accurate cycle counts for detailed performance measurements. OpenCCA provides a framework for the execution of Arm CCA code on currently available Arm hardware. The Performance Monitoring Unit (PMU) counter mechanisms are modified to enable measuring overhead across different exception levels. OpenCCA enables us to execute fast and accurate tests, which are reproducible by anyone for a relatively low budget and time investment [BS25].

Hardware Platform. As recommended by OpenCCA, we use the RADXA Rock5B as the underlying hardware. The available 16GB RAM is limited by the OpenCCA firmware emulation to just 4GB. The adaptability to 16GB is currently in progress. However, some of our unikernel Realm guests only function correctly on the stable release version. Therefore, the maximum amount of memory we can assign to a VM is 2GB. The firmware further prohibits the use of the 4 Cortex-A76 cores, restraining all execution to the 4 Cortex-A55 cores. When running a CVM, we pin the VMM to one core and isolate it from general-purpose scheduling using `isolcpus`. To measure the accurate number of instructions and cycles across execution levels, we employ the OpenCCA modified PMU.

Model and Dataset. The severe memory limitation of our VM to a maximum of 2GB forces us to use an extremely small LLM for our evaluation. Even minimal models with significant quality loss often exceed the maximum in RAM memory requirement. We therefore use a model of only 7.75MB size and cannot simulate qualitative inference [afr24]. For inference requests, we use randomly generated example texts.

System Configuration and Measurement. For all experiments, we choose a total payload size of 4MB divided into 256 slots. The size of the inference response is fixed to 4KB. To trigger measurements, even during the kernel bootup, we cause a `KVM_MMIO_EXIT` to the VMM. This requires a small addition to our Unikraft code, as well as a module for the Linux kernel. We modify Kvmtool to log the results when the exit points to a specific memory address.

To track KVM exit reasons, we use `bpftool` to trace the `kvm_exit` event. This provides more information on the Normal world than the Realm world, where the exit codes are not visible. To get the section-wise number of exits in the Realm, we trace the number of times the `handle_rec_exit` function is entered, using a hardware breakpoint.

For consistent values, we average 5 executions.

6.2 Realm versus Normal World Unikernel Performance

Research Question. Which system parts cause the highest decrease in performance in the Realm World versus the Normal World?

Metrics. We closely examine the performance of our custom Unikraft OS in the Realm and Normal world. We create a breakdown of the end-to-end inference service by measuring the time of individual sections and Realm-exclusive workflows. Furthermore, we examine the number of KVM exits and their origin to understand what creates Realm overhead.

Baseline. The baseline is the Unikraft VM without Arm CCA features, running in the Normal world.

6.2.1 Methodology and Measurements

We evaluate the start-to-finish execution of our CVM inference service (*End-to-End Execution*). Specifically, this measures the time between the Kvmtool main method entry and exit. We split this into three phases. The boot phase – initializing the VM until it is operational, the application phase – loading the model and replying to client requests using `llama.cpp` inference, and the shutdown phase – stopping VM execution when requested. For the end-to-end overview, we vary the number of inference requests and simultaneous clients, while the breakdown of the different phases is limited to one client and request. The breakdown displays metrics for different execution sections, which are explained in the following paragraphs.

Boot Phase. The boot phase is split into the initialization of the VM by the VMM, and the initialization done by the guest VM itself.

- *Kvmtool*: The complete hypervisor initialization is measured until the guest VM is started. This includes creating the VM and loading the kernel image (*Image Initialization*). In the Realm world, the guest setup must be finalized by securely populating the memory of the Realm (*VMM Realm Setup*).
- *Kernel*: The kernel initialization measures the time between starting the guest and the entry of the main application. In the Realm world, this includes the initialization of guest memory to its secure state (*Guest Realm Setup*).

Application Phase. The application phase includes the setup and execution of the secure inference service.

- *SHM Channel*: The shared memory communication channel is established. This includes the Realm guest changing the memory region to unprotected pages (*Realm Share Memory*).
- *Model Load*: The model owner requests loading the model. Measurement starts at the reception of the request and stops once the model is loaded and ready for inference.
- *Inference Service*: We measure the completion of one inference request. This includes the establishment of secure communication using the attestation protocol (*Secure Channel*), which includes the generation of an attestation report in Realm (*Realm Report*). To understand the overhead of pure llama.cpp inference, we use llama-bench with 32 input tokens and a context size of 512.

Shutdown Phase. The shutdown phase measures the time between the reception of a shutdown request and the Kvmtool main exit (*Shutdown*).

6.2.2 Results

Boot Phase. Figure 6.1 shows the breakdown of the VM bootup. The Realm guest must take an additional step to populate the kernel memory correctly. *VMM Realm Setup* produces an overhead of 0.19 seconds. However, the total boot time of the Realm guest is still lower than that of the Normal world VM. This is surprising, since an examination of the Realm boot process and previous research mandates the opposite. Examining the Kvmtool-specific VM initialization explains this phenomenon. When allocating the memory for a Realm guest, Kvmtool simply registers a `guest_memfd` but

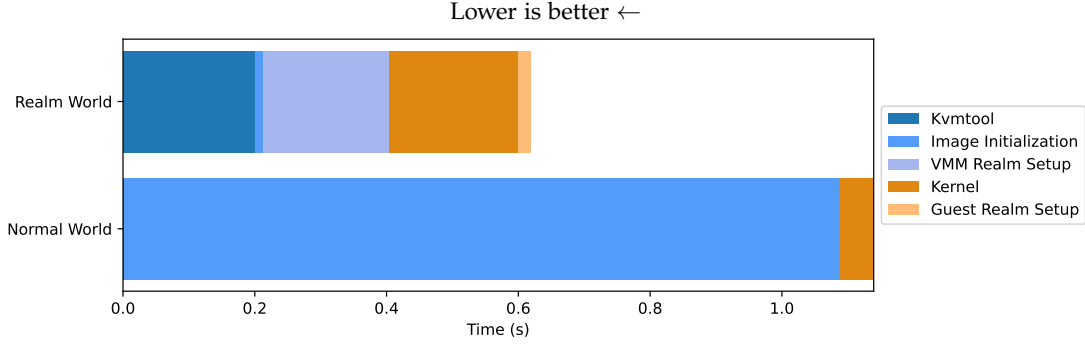


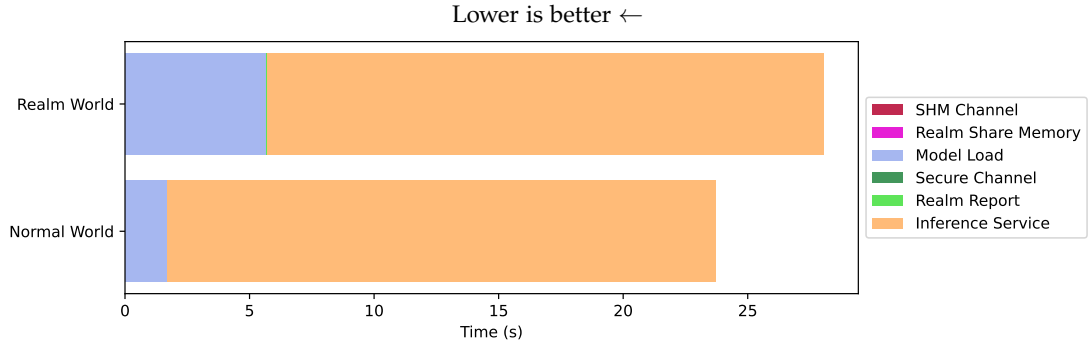
Figure 6.1: Comparison of Unikraft VM Boot Phase in Realm versus Normal World. Hypervisor sections in blue (*Kvmtool*), kernel sections in orange (*Kernel*).

does not back the file with any physical memory. On the other hand, a Normal VM immediately backs the `memfd` with allocated memory the size of the entire VM RAM. Therefore, the *Image Initialization* in Normal World takes 1.08 seconds, while the Realm equivalent only measures 0.01 seconds. The opposite can be observed during *Kernel* initialization, where the Realm guest takes about $3.8\times$ longer. Every access to a new physical guest address prompts a page fault. In the Normal world, this lazily builds the stage-2 page table. In the Realm world, additionally, the underlying physical pages must be allocated. Combined with the increased Realm exit overhead for about 3500 KVM exits (see Table 6.1), this slows down Realm initialization significantly. The setup of the Realm memory within the guest produces an overhead of 0.02 seconds.

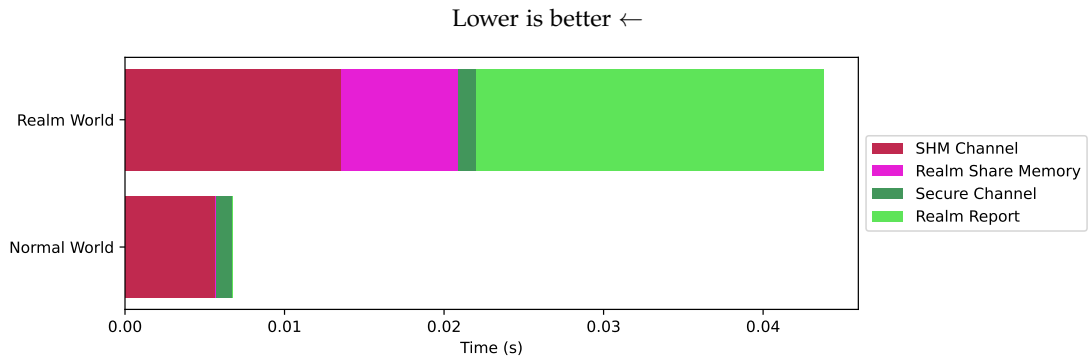
Application Phase. Figure 6.2a shows the breakdown of the application phase. *Model Load* is significantly faster in the Normal world, beating the Realm world by a factor of 3.2. Like the kernel initialization, this can be explained by the Realm overhead for KVM exits to build the stage-2 translation table and allocate physical pages. This is most severe during the loading process, where we can trace about 100 thousand exits (see Table 6.1). On the other hand, the difference in inference time is much smaller. This is to be expected, since less untouched memory will be used, leading to only about 10 thousand exits, incurring an overhead of about 0.36 seconds.

Our execution of `llama-bench` supports these findings. The load time in Realm (5.75s) far exceeds that of the Normal world (1.74s). On the other hand, the Realm (852tps) and Normal world (930tps) are much closer in throughput.

The establishment of the secure channel, sharing the memory, and the specific Realm instructions needed to support this are fast in comparison. Figure 6.2b shows that the execution of the appropriate RSI calls causes a minimal overhead.



(a) Complete Application Phase Containing All Sections.



(b) Fast Sections of the Application Phase, Invisible in the Complete Picture. Establishment of the SHM Channel in red and Secure Channel in green.

Figure 6.2: Comparison of Unikraft VM behavior in Realm versus Normal World.

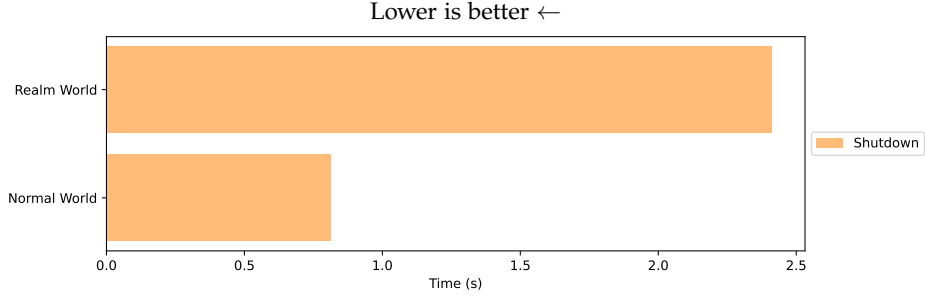
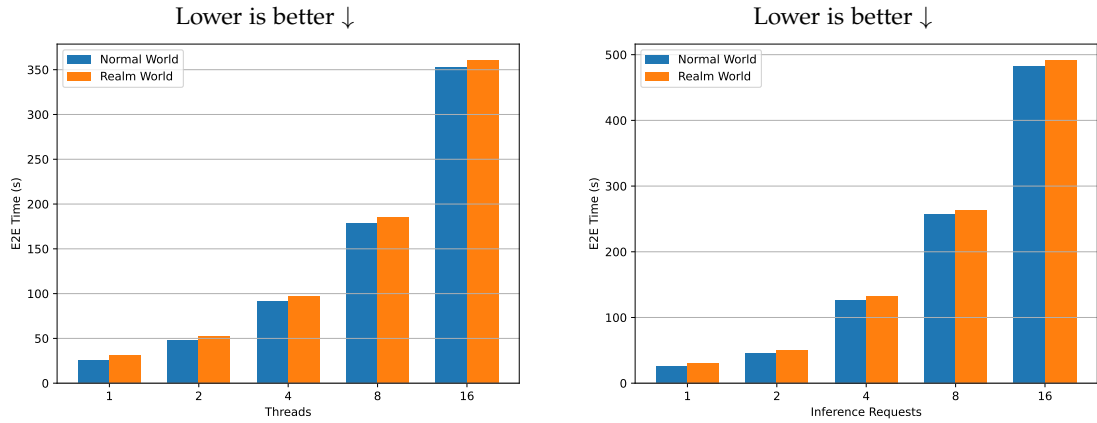


Figure 6.3: Comparison of Unikraft VM Shutdown Phase in Realm versus in Normal World.

Shutdown Phase. The shutdown (Figure 6.3) of the Realm guest exceeds that of the Normal world by a factor of 3. This is in line with our expectations since the host must destroy data granules, RTTs and RECs associated with the Realm to avoid leaking confidential data.

End-to-End Execution. As expected, the Normal world is consistently faster than the Realm World execution, though only by a small margin (Figure 6.4). With rising inference requests, there is a nearly linear increase in time. The total Realm world overhead decreases from 20% with one inference request to about 2% with 16 inference requests. Clearly, most of the performance penalty occurs in the boot phase, as well as the initial part of the application phase.



(a) End-to-End Execution. 1 Inference Request, X Clients. (b) End-to-End Execution. X Inference Requests, 1 Client.

Figure 6.4: End-to-end performance of Unikraft VM in Realm and Normal world.

Section	Normal World Exits (%)	Realm World Exits (%)
Kernel Initialization	3625.40 (2.99%)	3454.60 (2.86%)
↳ Realm Guest Setup	–	9.00 (0.01%)
SHM Channel	375.20 (0.31%)	416.40 (0.34%)
↳ Realm Share Memory	–	3.00 (0.00%)
Model Load	107266.80 (88.50%)	107116.80 (88.60%)
Inference Service	9911.00 (8.18%)	9886.40 (8.18%)
↳ Secure Channel	22.40 (0.02%)	9.20 (0.01%)
↳ Realm Report	–	5.00 (0.00%)
Shutdown	3.40 (0.00%)	3.80 (0.00%)
Total	121204.20 (100.00%)	120904.20 (100.00%)

Table 6.1: KVM Exit Distribution across Normal and Realm World Execution Phases.

VM Exits. Table 6.1 shows the number of KVM exits in each section. 99% of all exits are caused by guest faults. Due to our shared memory communication channel, only 0.5% of all exits are caused by MMIO emulation. The exit codes, only available in the Normal world, indicate that 95% of exits are caused by data aborts on unhandled memory. *Model Load* and *Inference Service* cause the most exits, since they produce the most memory accesses, which require either newly mapping or remapping the page table. The number of exits caused by Realm-specific instructions, for example, to share memory, is minimal in comparison.

6.3 Light-weight OS versus Full OS Performance

Research Question. Does the light-weight Unikraft OS exhibit improved performance versus a full OS such as the Linux kernel?

Metrics. We repeat a reduced breakdown of the previous section. Furthermore, we compare the Unikraft versus the Linux kernel size.

Baseline. The baseline is a Linux VM created using the OpenCCA Linux fork.

6.3.1 Methodology and Measurements

The limited Linux Realm guest support forces us to abandon guest-to-host communication. While sharing memory in kernel space is well supported in Linux, mapping this memory to user space is not trivial and must therefore be left to future work. Instead, we execute model loading and inference using an initial RAM disk to provide the model and inference requests. The measurements are the same as those in Section 6.2.1, though we remove all measurements concerning guest-to-host communication.

6.3.2 Results

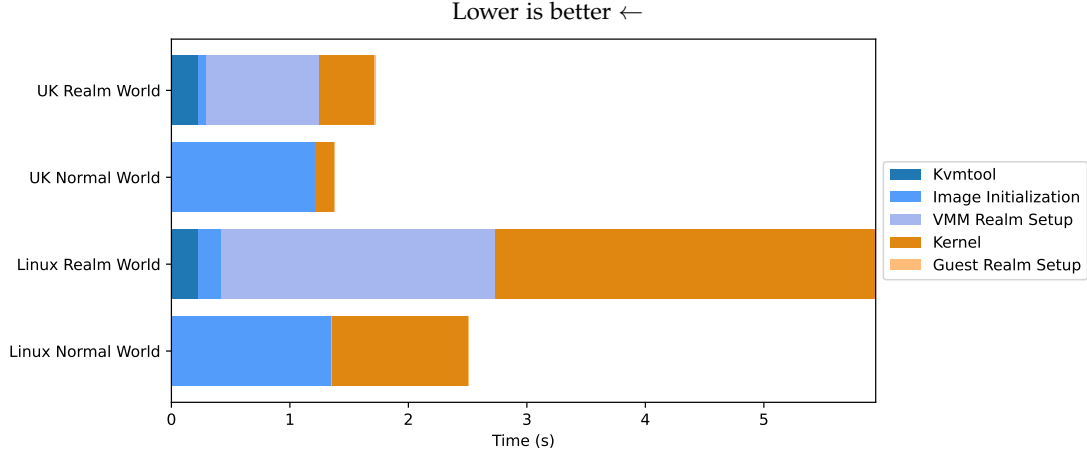


Figure 6.5: Unikraft versus Linux VM Boot Phase. Hypervisor sections in blue (*Kvmtool*), kernel sections in orange (*Kernel*).

Boot Phase. The boot phase (Figure 6.5) of the Linux kernel takes significantly longer than that of the Unikraft VM. In the Normal world, it is mainly the $7\times$ slower kernel initialization that causes the increase, due to the initialization of redundant devices. The modular Unikraft design allows us to completely eliminate this overhead.

The Realm world kernel initialization of the Linux VM takes $2.8\times$ longer than in the Normal world, whereas the Unikraft VM takes $2.9\times$ longer than its Realm counterpart. While the proportional increase is nearly identical, the difference in Normal world initialization time is magnified by the Realm world. This makes the Linux kernel initialization in Realm $6.7\times$ slower than the Unikraft equivalent.

The Linux hypervisor initialization slows down by a factor of 2 in the Realm whereas Unikraft only slows by a factor of 1.02. As we expected the larger image size heavily impacts the *VMM Realm Setup* section – the Linux image is 42.4MB large while the Unikraft image takes only 9MB of space.

In the Realm world, the entire boot phase takes about $3.5\times$ longer when using a Linux kernel, than when using our Unikraft OS.

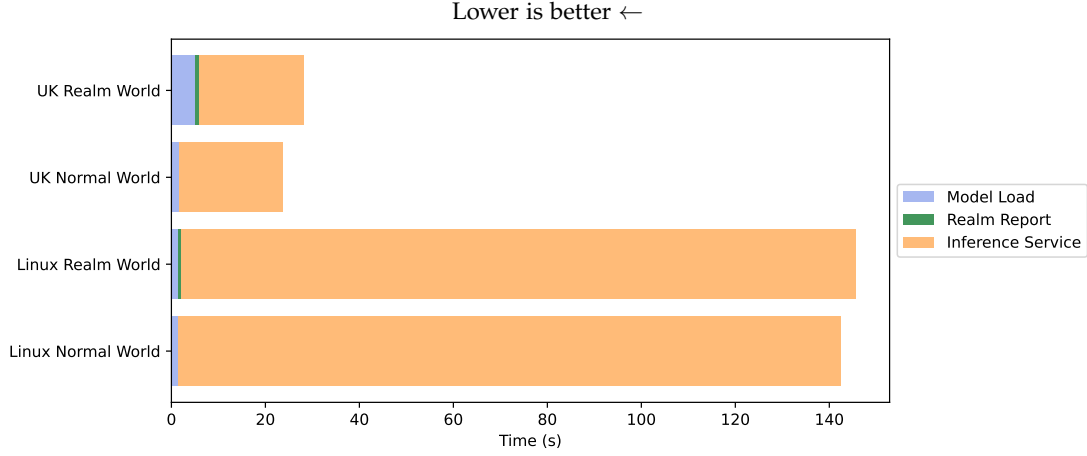


Figure 6.6: Unikraft versus Linux VM Application Phase. 1 Client and 1 Inference Request.

Application Phase. The comparison of the application phase is displayed in Figure 6.6. Linux model loading outperforms Unikraft model loading in the Realm World by a factor of 3.3. We assume that the longer initialization process of the Linux kernel has triggered the allocation of the necessary physical pages, while the Unikraft VM first faults during model load. The generation of the attestation report might take $1.6\times$ longer for the same reason, though further exploration is needed to see where the impact of page allocation ends. On the contrary, the generation of an inference response takes $6.3\times$ longer using a Linux VM. However, this increase can be seen both in the Normal and the Realm world. We suppose that a combination of kernel configuration and OpenCCA emulation causes this delay. Both Linux and Unikraft experience only a small amount of overhead during inference calculations.

6.4 Shared Memory Channel versus VirtIO Device Access

Research Question. Does the shared memory communication protocol offer performance advantages over VirtIO device usage in the Realm World?

Metrics. We measure the time it takes to perform memory-intensive communication using the shared memory channel versus a VirtIO mounted 9p file system within a Realm.

Baseline. The baseline is the Unikraft VM configured to use VirtIO 9p file sharing.

6.4.1 Methodology and Measurements

Since our Unikraft VirtIO 9p capabilities are rudimentary, we limit the experiments to loading files. We load the LLM model using the llama.cpp load function. Since our test environment is heavily limited in memory, this only allows us to test the load of the small 7.75MB model. We test more intensive communication by reading a 1GB text file into memory to be processed sequentially. The boot phase measurements conform to those explained in Section 6.2.1.

6.4.2 Results

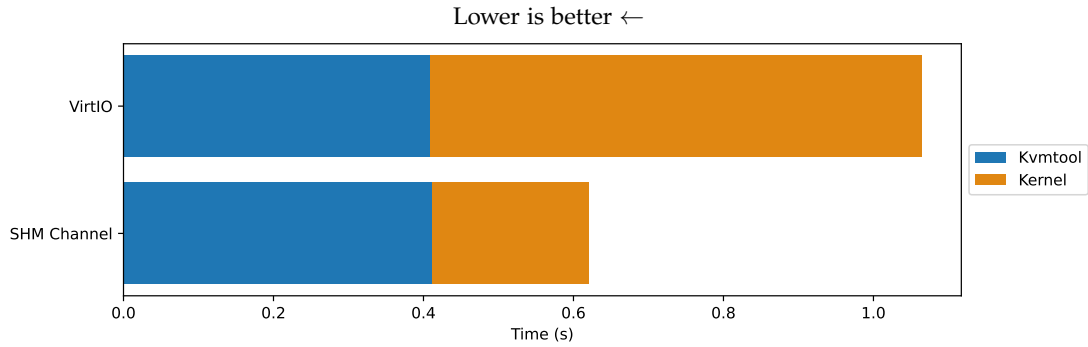


Figure 6.7: Comparison of Shared Memory versus VirtIO Unikraft VM Boot Phase in Realm.

Boot Phase. The difference in boot-time (Figure 6.7) shows that the setup and configuration of the VirtIO device adds a significant overhead to the VM kernel initialization. The initialization of the Kernel takes about $3.25\times$ longer when configuring VirtIO.

Phase	VirtIO	Shared Memory	Difference (%)
Model Load	114004.80	107130.20	-6.03%
File Load	2970628.20	480.00	-99.98%

Table 6.2: Comparison of Number of KVM Exits between VirtIO and Shared Memory.

Model Load. The llama.cpp model load requires loading from a file, instead of directly from memory. This requires our shared memory approach to write the model into a new file before model load can begin. Still, shared memory outperforms VirtIO by a factor of 1.17 (Figure 6.8a). This VirtIO performance loss has two causes. Firstly, llama.cpp uses a default allocator that cannot be easily configured. To ensure access to

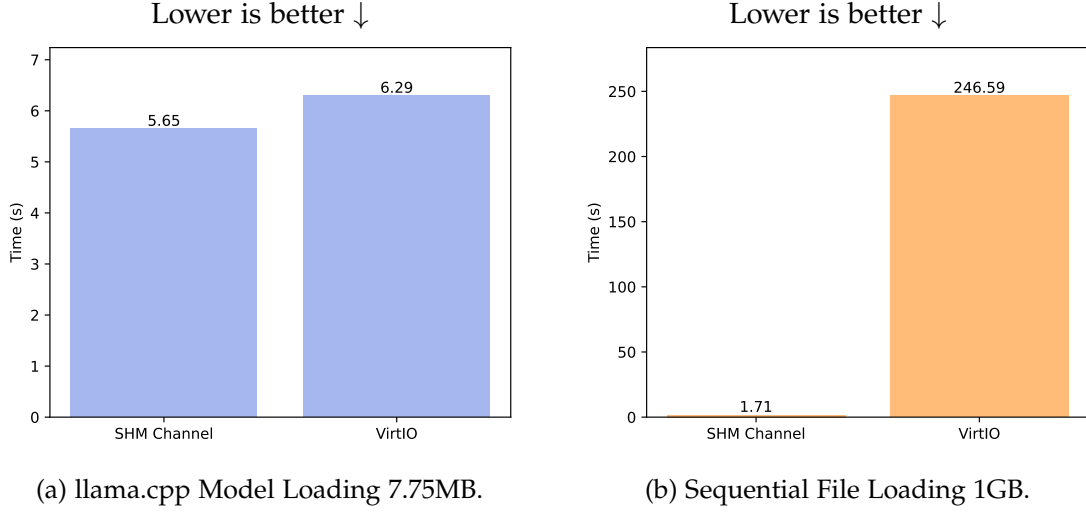


Figure 6.8: Shared Memory versus VirtIO Unikraft VM Load Times in Realm.

the 9p device, we must therefore buffer all read operations using the shared allocator. This copying significantly slows down file access. Secondly, the shared memory channel avoids the MMIO emulation KVM exits, lowering the total number of world switches by 6% (Table 6.2).

File Load. When the file can be incrementally loaded and sequentially worked on, the shared memory channel significantly outperforms the VirtIO by a factor of about 145 (Figure 6.8b). The addition of the para-virtualized device layer and world switches causes a large amount of overhead. Table 6.2 shows that the number of KVM exits is decreased by 99.98% when using the shared memory channel. However, access to a file shared using VirtIO is far more flexible and requires less development effort. Both communication channels are therefore vital for a unikernel design that is both adaptable and performant.

7 Related Work

Secure LLM Inference. The field of LLM inference security addresses a multitude of different attack vectors. Security concerns include targeting the model itself, for example, via prompt injection or gradient leakage [DAW25]. Narrowing the scope to privacy and confidentiality-enhancing technologies, there are alternatives to the hardware-based CC approach we explored [And+26]. Secure Multi-Party Computation and Fully Homomorphic Encryption techniques for confidential inference are constantly being improved upon [Tho+25; Moo+24]. TEE based concepts are designed to fit the industry deployment demands. A recent paper shows that Intel TDX and SGX can support secure inference with an acceptable overhead [Chr+25]. AICore uses isolation to provide secure end-user device inference on smartphones [Goo26]. Secure accelerator access remains one of the biggest challenges in the field. PipeLLM proposes an efficient speculative encryption methodology [Abd+25]. Others suggest the use of an accelerator-based TEE, decoupling security from the CPU [Vas+23; Pen+25].

Arm CCA Extensions. Secure accelerator access is already a big research topic for Arm CCA. RMM version 2.1 is currently available in draft form and describes the assignment, attestation, and secure access of devices to Realm-owned memory [ARM26]. This approach enables secure device pass-through, instrumentalizing device attestation mediated by the RMM to create a secure channel according to the device protection model. Overwhelmingly, this means traffic protection using the Integrity and Data Encryption (IDE) standard. Portal presents an approach to pass-through device assignment that eliminates the encryption overhead on System on Chips (SoCs). It provides an Arm CCA hardware-isolated plaintext memory region, by instrumentalizing the System Memory Management Unit (SMMU) and GPC [San+25]. ACAI specifically protects accelerator access using hardware-level encryption and memory isolation of the device [Sri+24]. CAGE secures GPU use without encryption. To mitigate performance overhead, a shadow task mechanism is employed to only execute the minimum number of required operations in isolated memory while protecting data [Wan+26]. Outside of device access, SHELTER presents an extension to provide isolation for Normal world applications [Zha+23]. NanoZone adds isolation levels against intra-VM adversaries [Liu+25]. Instead of our unikernel approach, RContainer provides an extension to isolate containers from each other and the OS [Zho+25].

Secure Unikernel Execution. An examination of unikernels for Arm-powered edge computing has shown they offer improved resource efficiency and rapid scaling on Arm devices [KTK24]. Talbot, Pikula, et al. [Tal+20] conducted a security investigation and praised the minimized attack surface and lack of shell, but pointed out critical missing features at the heart of unikernel vulnerabilities. This has prompted further research into security measures for unikernels. Wollman and Hastings [WH26] suggests an ASLR-like increase in randomization for the OSv unikernel. UniGuard leverages Intel SGX to protect sensitive computations in an enclave within the MirageOS unikernel, while Graphene-SGX provides a secured libOS [SG18; TPV17]. To extend protection past the threat model of CC, INCOGNITOS provides a self-obfuscating unikernel design to guard against side-channel attacks [Duy+25].

8 Conclusion

This work supplies the basis for a fully Arm CCA compatible unikernel. We provide required Realm guest functionality as well as a proof-of-work for shared device VirtIO access. This allowed us to conduct the first comparative evaluation of unikernel versus full OS performance within an Arm CCA Realm. The use of a unikernel improves the boot-up by a factor of 3.5 as well as the inference by a factor of 6.3.

Under the larger scope of the LLM-OS project, we provide the design of an end-to-end inference service secured through hardware-based confidential computing. The integration of Arm CCA capabilities into Unikraft leads to a small and performant CVM. The TEE protects both model and user data from other processes as well as privileged entities, such as the hypervisor and OS. We leverage the hardware attestation to ensure both secure execution and communication with outside entities. This required the design of a shared memory communication channel to enable efficient and adaptable message exchanges with the host. Loading a small model improves by a factor of 1.2, and we show that this improvement grows for large file transmissions. Using the OpenCCA framework, our evaluation provides accurate performance metrics that will inform future iterations on the design. The modular nature of the unikernel, as well as its compatibility with various development environments, allows easy extensibility in the future. We consider possible avenues for our system based on our results.

9 Future Work

In this thesis, we have created the basic support for an Arm CCA unikernel, which can be extended to support more features in the future. While our secure channel design offers adaptable and high-performance communication options, the development effort severely exceeds that of common devices. To enable stable, well-supported access methods, VirtIO support is a must. This addition requires either an efficient bounce-buffer implementation or secure device assignment once available in development environments.

Furthermore, we have presented an end-to-end design for secure LLM inference. This too can serve as the basis for future improvements and extensions. Due to resource limitations, our implementation and evaluation were limited. Extending the OpenCCA framework to include more cores and memory will make results more accurate for real-life deployment. Gauging the performance impact on cloud computing requires the integration of remote communication. To support remote entities, robust network communication must be implemented. Our simple attestation protocol can be extended to TLS with remote attestation [Sar+24].

Due to the limitations of secure device access, our system is limited to CPU-based inference. Efficient LLM inference requires accelerator access. In the future, this can be supported through secure device pass-through. However, pass-through devices require full device driver support within the unikernel guest. As an alternative approach, vAccel uses API-remoting to move the driver outside of the unikernel [Kam26]. However, both the communication and VMM plugin in its design reside in the Normal world. One topic of future exploration could be the isolation of these components using an approach like SHELTER [Zha+23]. Another option is designed with Arm CCA compatibility already in mind. CAGE moves the driver into the Normal world and employs a shadow task mechanism to execute a minimum number of operations in isolated memory [Wan+26]. Both of these approaches require significant changes to the Arm CCA TCB and therefore must be examined carefully.

To expand on the on-device use case, the isolation of the client application from potentially malicious Normal world processes is a further avenue of research.

Acronyms

API Application Programming Interface. 26–28

Arm CCA Arm Confidential Compute Architecture. iv, 2–4, 7–9, 11, 14–17, 19, 20, 23, 24, 27, 32, 33, 43, 45, 46

CC confidential computing. iv, 1–7, 43, 44

CSP Cloud Service Provider. 5, 7

CVM Confidential Virtual Machine. 1–4, 7, 8, 11, 17, 32, 33, 45

DA Device Assignment. 11

DH Diffie-Hellmann. 20

DMA Direct Memory Access. 1, 14, 26–28

FVP Fixed Virtual Platform. 23, 32

GPC Granule Protection Check. 9, 43

GPT Granule Protection Table. 9

GPU Graphics Processing Unit. 3, 31, 43

IDE Integrity and Data Encryption. 43

IOMMU IO Memory Management Unit. 26, 27

IPA intermediate physical address. 9, 25, 26

KVM Kernel-based Virtual Machine. 23, 26, 33, 35, 38, 41, 42, 51

libOS library OS. iv, 12, 14–17, 26, 44

LLM Large Language Model. iv, 2–4, 6, 13–17, 22, 28, 31, 32, 41, 43, 45, 46

MMIO Memory-Mapped IO. 9, 15, 18, 19, 27, 38, 42

MMU Memory Management Unit. 9

MRD Memory Region Descriptor. 25

MSB Most Significant Bit. 9

NSE Non-Secure Enable. 9, 25–27

OS Operating System. iv, 2, 3, 6, 12, 15, 17, 24, 31, 33, 38, 39, 45

PA Physical Address. 9

PAS physical address space. 7–9

PE Processing Element. 7

PMU Performance Monitoring Unit. 32

RAK Realm Attestation Key. 11

RAM Random Access Memory. 9, 25, 28, 31, 32, 38

RATS Remote Attestation Specification. 11, 20

REC Realm Execution Context. 11, 37

REM Realm Extended Measurement. 11, 20

RHI Realm Host Interface. 8

RIM Realm Initial Measurement. 9, 11, 20

RME Realm Management Extension. 7, 11, 23, 24

RMI Realm Management Interface. 8

RMM Realm Management Monitor. 7–9, 17, 24, 43

RPV Realm Personalization Value. 24

RSI Realm Service Interface. 8, 9, 11, 14, 17, 24–26, 28, 35

RTT Realm Translation Table. 9, 37

SEV Secure Execution Virtualization. 2

SGX Software Guard eXtensions. 2, 43, 44

SMC Secure Monitor Call. 24

SMMU System Memory Management Unit. 43

SMPC Secure Multi-Party Computation. 5

SNP Secure Nested Paging. 2

SoC System on Chip. 43

TCB Trusted Computing Base. 2, 14, 19, 46

TDX Trust Domain eXtensions. 2, 43

TEE trusted execution environment. iv, 2, 5–7, 15, 43, 45

TTFT Time To First Token. 17

VA Virtual Address. 9

VirtIO Virtual Input-Output. 4, 14, 15, 18, 19, 26–28, 31, 40–42, 45, 46, 50

VM Virtual Machine. 1, 2, 9, 11, 12, 15, 17, 18, 20, 29, 32–41, 43, 50

VMM Virtual Machine Monitor. 14, 16, 17, 19, 23–28, 32–34, 46

List of Figures

2.1	Overview of Confidential Computing Environment and Participants. Figure informed by [Fen+24].	6
2.2	Arm CCA Overview. Figure informed by [Arm].	7
2.3	Realm Creation and Realm Exit Workflows. Figure informed by [Arm24].	10
2.4	Comparison of Traditional Virtual Machine and Unikernel Software Layers. Figure informed by [Goe+18] and [Mad+13a].	12
3.1	System Overview.	13
4.1	Attestation and Key Exchange Protocol.	21
5.1	Prototype Overview.	22
5.2	VirtIO Device Access	27
5.3	Shared Memory Communication.	29
5.4	Inference Service Communication Protocol.	30
6.1	Comparison of Unikraft VM Boot Phase in Realm versus Normal World. Hypervisor sections in blue (<i>Kvmtool</i>), kernel sections in orange (<i>Kernel</i>).	35
6.2	Comparison of Unikraft VM behavior in Realm versus Normal World.	36
6.3	Comparison of Unikraft VM Shutdown Phase in Realm versus in Normal World.	37
6.4	End-to-end performance of Unikraft VM in Realm and Normal world.	37
6.5	Unikraft versus Linux VM Boot Phase. Hypervisor sections in blue (<i>Kvmtool</i>), kernel sections in orange (<i>Kernel</i>).	39
6.6	Unikraft versus Linux VM Application Phase. 1 Client and 1 Inference Request.	40
6.7	Comparison of Shared Memory versus VirtIO Unikraft VM Boot Phase in Realm.	41
6.8	Shared Memory versus VirtIO Unikraft VM Load Times in Realm.	42

List of Tables

2.1	Access control between worlds.	8
6.1	KVM Exit Distribution across Normal and Realm World Execution Phases.	38
6.2	Comparison of Number of KVM Exits between VirtIO and Shared Memory.	41

Bibliography

- [Abd+25] S. Abdollahi, A. A. Sadi, M. Kogias, D. Kotz, and H. Haddadi. *Confidential, Attestable, and Efficient Inter-CVM Communication with Arm CCA*. 2025. arXiv: 2512.01594 [cs.CR].
- [afr24] afrideva. *afrideva/Tinystories-gpt-0.1-3m-GGUF at main* — *huggingface.co*. <https://huggingface.co/afrideva/{T}inystories-gpt-0.1-3m-{G}{G}{U}{F}/tree/main>. [Accessed 24-04-2026]. 2024.
- [Amb24] S. Ambiel. *The Case for Confidential Computing: Delivering Business Value Through Protected, Confidential Data Processing*. 2024. doi: 10.70828/iynl6589.
- [And+26] D. Andreoletti, A. Rudi, E. Carpanzano, F. Lelli, and T. Leidi. *Privacy-Preserving LLM Inference in Practice: A Comparative Survey of Techniques, Trade-Offs, and Deployability*. Cryptology ePrint Archive, Paper 2026/105. 2026.
- [Arm] Arm. *Arm confidential compute architecture*. <https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture>. [Accessed 27-04-26].
- [Arm21] Arm. *Arm CCA Security Model 1.0*. <https://developer.arm.com/documentation/DEN0096/latest/>. Version A.a. Date of issue: 02/08/2021. Aug. 2021.
- [Arm24] Arm Limited. *Realm Management Monitor Specification*. <https://developer.arm.com/documentation/den0137/1-0rel0/?lang=en>. Version 1.0-rel0. Document quality: REL. Non-confidential. Arm Limited, 2024.
- [Arm25a] Arm Limited. *Learn the architecture - Arm Confidential Compute Architecture software stack*. <https://developer.arm.com/documentation/den0127/300/Software-components>. Version 3.0. Non-Confidential. Issue den0127_300_06_en. Copyright © 2022–2023, 2025 Arm Limited (or its affiliates). Arm Limited, 2025.
- [Arm25b] Arm Limited. *Learn the Architecture: Introducing Arm Confidential Compute Architecture*. <https://developer.arm.com/documentation/den0125/400/Arm-CCA-Software-Architecture>. Version 4.0. Non-Confidential. Issue den0125_400_en. Copyright © 2022–2023, 2025 Arm Limited (or its affiliates). Arm Limited, 2025.

- [ARM26] ARM. *Realm Management Monitor specification DEN0137 2.0-beta1*. <https://developer.arm.com/documentation/den0137/2-0beta1/>. [Accessed 19-04-2026]. 2026.
- [Bel05] F. Bellard. "QEMU: A Fast and Portable Dynamic Translator." In: *Proceedings of the USENIX Annual Technical Conference (USENIX ATC)*. Anaheim, CA, USA: USENIX Association, Apr. 2005.
- [Ben] A. Bennee. *Building an RME stack for QEMU*. <https://linaro.atlassian.net/wiki/spaces/QEMU/pages/29051027459/Building+an+RME+stack+for+QEMU+with+the+OP+TEE+build+environment>. [Accessed 01-02-2026].
- [BS25] A. Bertschi and S. Shinde. "OpenCCA: An Open Framework to Enable Arm CCA Research." In: *8th Workshop on System Software for Trusted Execution (SysTEX 2025)*. 2025.
- [Bue25] P. Bues. *Unlocking the Future of Data Security: Confidential Computing as a Strategic Imperative*. <https://confidentialcomputing.io/wp-content/uploads/sites/10/2025/11/US53866125.pdf>. Nov. 2025.
- [Che25] A. Cherney Max A; Bajwa. *Arm shares drop as outlook disappoints; company looks to invest to make own chips*. https://www.reuters.com/business/retail-consumer/arm-shares-drop-outlook-disappoints-company-looks-invest-make-own-chips-2025-07-30/?utm_source=chatgpt.com. [Accessed 03-05-2026]. 2025.
- [Chr+25] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler. *Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs*. 2025. arXiv: 2509.18886 [cs.PF].
- [Com] T. K. D. Community. *The linux kernel*. <https://docs.kernel.org/arch/arm64/arm-cca.html>. [Accessed 27-04-2026].
- [DAW25] B. C. Das, M. H. Amini, and Y. Wu. "Security and Privacy Challenges of Large Language Models: A Survey." In: *ACM Comput. Surv.* 57.6 (Feb. 2025). ISSN: 0360-0300. DOI: 10.1145/3712001.
- [DL21] O. Demigha and R. Larguet. "Hardware-based solutions for trusted cloud computing." In: *Computers & Security* 103 (2021), p. 102117. ISSN: 0167-4048. DOI: <https://doi.org/10.1016/j.cose.2020.102117>.
- [Duy+25] K. D. Duy, J. Kim, H. Lim, and H. Lee. "INCOGNITOS: A Practical Unikernel Design for Full-System Obfuscation in Confidential Virtual Machines." In: *2025 IEEE Symposium on Security and Privacy (SP)*. 2025, pp. 4192–4209. DOI: 10.1109/SP61157.2025.00222.

- [Enb+nd] P. Enberg, C. Gorcunov, A. He, S. Levin, P. Joshi, et al. *kvmtool: Native Linux KVM Tool*. <https://github.com/kvmtool/kvmtool>. Accessed: 2026-04-17. n.d.
- [Fen+24] D. Feng, Y. Qin, W. Feng, W. Li, K. Shang, and H. Ma. "Survey of research on confidential computing." In: *IET Communications* 18.9 (2024), pp. 535–556. DOI: <https://doi.org/10.1049/cmu2.12759>. eprint: <https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/cmu2.12759>.
- [Fou23] T. L. Foundation. *About the Confidential Computing Consortium*. <https://confidentialcomputing.io/about/>. [Accessed 10-03-2026]. 2023.
- [Git] A. Gitlab. *Obtain the Armv-A Base RevC AEM FVP*. <https://gitlab.arm.com/arm-reference-solutions/arm-reference-solutions-docs/-/blob/master/docs/aemfvp-a-rme/install-fvp.rst>. [Accessed 01-05-2026].
- [Goe+18] T. Goethals, M. Sebrechts, A. Atrey, B. Volckaert, and F. De Turck. "Unikernels vs Containers: An In-Depth Benchmarking Study in the Context of Microservice Applications." In: *2018 IEEE 8th International Symposium on Cloud and Service Computing (SC2)*. 2018, pp. 1–8. DOI: 10.1109/SC2.2018.00008.
- [Goo26] Google. *Gemini Nano*. <https://developer.android.com/ai/gemini-nano>. [Accessed 27-04-26]. 2026.
- [Has+15] I. A. T. Hashem, I. Yaqoob, N. B. Anuar, S. Mokhtar, A. Gani, and S. Ullah Khan. "The rise of "big data" on cloud computing: Review and open research issues." In: *Information Systems* 47 (2015), pp. 98–115. ISSN: 0306-4379. DOI: <https://doi.org/10.1016/j.is.2014.07.006>.
- [Hua+24] H. Huang, F. Zhang, S. Yan, T. Wei, and Z. He. "SoK: A Comparison Study of Arm TrustZone and CCA." In: *2024 International Symposium on Secure and Private Execution Environment Design (SEED)*. 2024, pp. 107–118. DOI: 10.1109/SEED61283.2024.00021.
- [Kam26] S. Kammermeier. *A Unikernel-based OS for On-Device LLM Inference*. [Master Thesis TUM DSE]. 2026.
- [KTK24] S. Kaiser, A. S. Tosun, and T. Korkmaz. "Exploring the viability of unikernels for ARM-powered edge computing." In: (Dec. 2024). arXiv: 2412.03030 [cs.DC].

- [Kue+21] S. Kuenzer, V.-A. Bădoiu, H. Lefeuvre, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, Ș. Teodorescu, C. Răducanu, C. Banu, L. Mathy, R. Deaconescu, C. Raiciu, and F. Huici. “Unikraft: fast, specialized unikernels the easy way.” In: *Proceedings of the Sixteenth European Conference on Computer Systems*. EuroSys ’21. ACM, Apr. 2021, pp. 376–394. doi: 10.1145/3447786.3456248.
- [Kuz23] V. Kuznetsov. *Introduction to confidential Virtual Machines*. <https://www.redhat.com/en/blog/introduction-confidential-virtual-machines>. June 2023.
- [Liu+25] S. Liu, Y. Gao, M. Zhang, and J. Wang. *NanoZone: Scalable, Efficient, and Secure Memory Protection for Arm CCA*. 2025. arXiv: 2506.07034 [cs.CR].
- [Mad+13a] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott, B. Singh, T. Gazagnaire, S. Smith, S. Hand, and J. Crowcroft. “Unikernels: library operating systems for the cloud.” In: *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS ’13. Houston, Texas, USA: Association for Computing Machinery, 2013, pp. 461–472. isbn: 9781450318709. doi: 10.1145/2451116.2451167.
- [Mad+13b] A. Madhavapeddy, R. Mortier, C. Rotsos, D. Scott, B. Singh, T. Gazagnaire, S. Smith, S. Hand, and J. Crowcroft. “Unikernels: library operating systems for the cloud.” In: *SIGARCH Comput. Archit. News* 41.1 (Mar. 2013), pp. 461–472. issn: 0163-5964. doi: 10.1145/2490301.2451167.
- [Mas+25] N. Maslej, L. Fattorini, R. Perrault, Y. Gil, V. Parli, N. Kariuki, E. Capstick, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald, T. Walsh, A. Hamrah, L. Santarlasci, J. B. Lotufo, A. Rome, A. Shi, and S. Oak. *Artificial Intelligence Index Report 2025*. 2025. arXiv: 2504.07139 [cs.AI].
- [Moo+24] J. Moon, D. Yoo, X. Jiang, and M. Kim. *THOR: Secure Transformer Inference with Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2024/1881. 2024. doi: 10.1145/3719027.3765150.
- [OAS23] OASISOpen. *Virtual I/O Device (VIRTIO) Version 1.3* — [docs.oasis-open.org](https://docs.oasis-open.org/virtio/virtio/v1.3/csd01/virtio-v1.3-csd01.html). <https://docs.oasis-open.org/virtio/virtio/v1.3/csd01/virtio-v1.3-csd01.html>. [Accessed 10-03-2026]. 2023.
- [Pen+25] X. Peng, Y. Yang, S. Pan, J. Huang, Y. Liang, H. Lu, F. Zhang, and X. Li. “SecNPU: Securing LLM Inference on NPU.” In: *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. 2025, pp. 25–32. doi: 10.1109/ICCD65941.2025.00011.

- [RYL25] J. Ren, H. Yang, and Y. Liao. “BlazeVMM: Fast Startup of ARM CCA MicroVMs for Confidential Serverless Computing.” In: *Proceedings of the 4th International Conference on Computer, Artificial Intelligence and Control Engineering*. CAICE '25. Association for Computing Machinery, 2025, pp. 859–863. ISBN: 9798400712647. DOI: 10.1145/3727648.3727789.
- [San+25] F. Sang, J. Lee, X. Zhang, and T. Kim. “Portal: Fast and Secure Device Access with Arm CCA for Modern Arm Mobile System-on-Chips (SoCs).” In: *2025 IEEE Symposium on Security and Privacy (SP)*. 2025, pp. 4099–4116. DOI: 10.1109/SP61157.2025.00013.
- [Sar+24] M. U. Sardar, T. Fossati, S. Frost, and S. Xiong. “Formal Specification and Verification of Architecturally-Defined Attestation Mechanisms in Arm CCA and Intel TDX.” In: *IEEE Access* 12 (2024), pp. 361–381. DOI: 10.1109/ACCESS.2023.3346501.
- [SG18] I. Sfyarakis and T. Gross. “UniGuard: Protecting Unikernels Using Intel SGX.” In: *2018 IEEE International Conference on Cloud Engineering (IC2E)*. 2018, pp. 99–105. DOI: 10.1109/IC2E.2018.00032.
- [Sib+24] S. Siby, S. Abdollahi, M. Maheri, M. Kogias, and H. Haddadi. *GuaranTEE: Towards Attestable and Private ML with CCA*. 2024. arXiv: 2404.00190 [cs.CR].
- [Sri+24] S. Sridhara, A. Bertschi, B. Schlüter, M. Kuhne, F. Aliberti, and S. Shinde. “ACAI: Protecting Accelerator Execution with Arm Confidential Computing Architecture.” In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3423–3440. ISBN: 978-1-939133-44-1.
- [Sta+24] R. Staab, M. Vero, M. Balunović, and M. Vechev. *Beyond Memorization: Violating Privacy Via Inference with Large Language Models*. 2024. arXiv: 2310.07298 [cs.AI].
- [Tal+20] J. Talbot, P. Pikula, C. Sweetmore, S. Rowe, H. Hindy, C. Tachtatzis, R. Atkinson, and X. Bellekens. “A Security Perspective on Unikernels.” In: *2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*. 2020, pp. 1–7. DOI: 10.1109/CyberSecurity49315.2020.9138883.
- [Tho+25] R. K. Thomas, L. Zahran, E. Choi, A. Potti, M. Goldblum, and A. Pal. “Hidden No More: Attacking and Defending Private Third-Party LLM Inference.” In: *Forty-second International Conference on Machine Learning*. 2025.

- [TPV17] C.-c. Tsai, D. E. Porter, and M. Vij. “Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX.” In: *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. Santa Clara, CA: USENIX Association, July 2017, pp. 645–658. ISBN: 978-1-931971-38-6.
- [Uni] Unikraft. *unikraft/drivers/ukrandom/lcpu/README.md at staging · unikraft/unikraft* — *github.com*. <https://github.com/unikraft/unikraft/blob/staging/drivers/ukrandom/lcpu/{R}{E}{A}{D}{M}{E}.md>. [Accessed 18-04-2026].
- [Üru25] B. E. Ürun. *LLM-OS: Orchestrating Edge Inference with Priority Scheduling and Adaptive KV Cache Management*. [Master Thesis TUM DSE]. 2025.
- [Vas+23] K. Vaswani, S. Volos, C. Fournet, A. N. Diaz, K. Gordon, B. Vembu, S. Webster, D. Chisnall, S. Kulkarni, G. Cunningham, R. Osborne, and D. Wilkinson. “Confidential Computing within an AI Accelerator.” In: *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, July 2023, pp. 501–518. ISBN: 978-1-939133-35-9.
- [Wan+25] W. Wang, K. Li, B. Ji, X. Liu, J. Yu, and Q. Wu. “A Survey of AI Inference Technologies for On-Device Systems.” In: *IEEE Internet of Things Journal* 12.24 (2025), pp. 51927–51950. doi: 10.1109/JIOT.2025.3613691.
- [Wan+26] C. Wang, K. Lu, F. Zhang, Y. Deng, K. Leach, J. Cao, Z. Ning, S. Yan, T. Wei, and Z. He. “Building Confidential Accelerator Computing Environment for Arm CCA.” In: *IEEE Transactions on Dependable and Secure Computing* 23.1 (2026), pp. 1308–1325. doi: 10.1109/TDSC.2025.3615787.
- [WH24] A. Wollman and J. Hastings. “A Survey of Unikernel Security: Insights and Trends from a Quantitative Analysis.” In: *2024 Cyber Awareness and Research Symposium (CARS)*. 2024, pp. 1–9. doi: 10.1109/CARS61786.2024.10778787.
- [WH26] A. Wollman and J. Hastings. “Hardening the OSv Unikernel With Efficient Address Randomization: Design and Performance Evaluation.” In: *2026 14th International Symposium on Digital Forensics and Security (ISDFS)*. 2026, pp. 1–6. doi: 10.1109/ISDFS69419.2026.11459124.
- [Xu+24] X. Xu, W. Wang, Y. Wu, C. Wang, H. Zhu, H. Ma, Z. Min, Z. Pang, R. Hou, and Y. Jin. *virtCCA: Virtualized Arm Confidential Compute Architecture with TrustZone*. 2024. arXiv: 2306.11011 [cs.CR].

- [ZA25] S. Zobaed and M. Amini Salehi. "Confidential Computing Across Edge-To-Cloud for Machine Learning: A Survey Study." In: *Software: Practice and Experience* 55.5 (2025), pp. 896–924. DOI: <https://doi.org/10.1002/spe.3398>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.3398>.
- [Zha+23] Y. Zhang, Y. Hu, Z. Ning, F. Zhang, X. Luo, H. Huang, S. Yan, and Z. He. "SHELTER: Extending Arm CCA with Isolation in User Space." In: *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 6257–6274. ISBN: 978-1-939133-37-3.
- [Zha23] X. Zhang. *GSOC'23: ARM CCA support for Unikraft (part 1)*. <https://unikraft.org/blog/2023-06-23-unikraft-gsoc-arm-cca-1>. 2023.
- [Zho+25] Q. Zhou, W. Cao, X. Jia, P. Liu, S. Zhang, J. Chen, S. Xu, and Z. Song. "RContainer: A secure container architecture through extending ARM CCA hardware primitives." In: *Proceedings 2025 Network and Distributed System Security Symposium*. San Diego, CA, USA: Internet Society, 2025.